# Algorithm

An algorithm is a finite set of instructions that accomplishes a particular task

# Properties of an Algorithm

- Input: Zero or more quantities are supplied externally.
- Output: Atleast one quantity is produced.
- Definiteness: Each instruction is clear and unambiguous
- Finiteness: For all cases, algorithm terminates after a finite number of steps
- Effectiveness: Each instruction must be very basic.

- Algorithms that are definite and effective are also called computational procedures.
- A program is the expression of an algorithm in a programming language.
- Active areas in study of algorithms:
  - How to devise algorithms: Certain design techniques are available.
  - How to validate algorithms: To show that algorithm produces correct answer for all possible legal inputs independently of the issues concerning programming language. Once an algorithm is validated, program is written and verified.
  - How to analyze algorithms: Determining how much computing time and storage an algorithm requires. Helps in making quantitative judgments about one algorithm over the other. Helps in predicting whether software will meet efficiency constraints?
  - How to test a program: Two phases: debugging and profiling (performance measurement)

# Algorithm Specification

- Algorithm specification using pseudo code that resembles C.
  - Comments begin with //
  - Block is represented by braces { }
  - Identifier begins with a letter. Simple data types are assumed. Compound data types can be formed using record.

```
node =  record
        { datatype_1 data1;
          …
          datatype_n datan;
          node* link;
        }
```

    . And -> can be used to denote the fields of type node.
  - Assignment := operator is used to assign values.
  - Two boolean values are assumed **true** and **false**. Logical and relative operators are provided.

- Multidimensional array is accessed as A[i,j]
- Looping constructs for, while and repeat until are provided.
  - for i:= 1 to 10 step 1 do {.....}
  - while i<10 do { … }
  - Repeat {.....} until (i>10)
- Break statement results in exit from innermost loop.
- Return statement results in exit of the function.
- Conditional statements if and case are provided.
  - if (a>b) then c:=a else c:=b
  - case
    { :a=1: c:=10;
    :a=2: c:=20;
    :else: c:=100;
    }
- Input and output is performed using read and write statements respectively.
- Only one type of procedure 'Algorithm' is provided having a heading and a body.
- Simple variables are passed by value. Arrays are passed by reference.

- Example: Selection sort algorithm.
  - for i:=1 to n do
    {
    Examine a[i] to a[n] and suppose the smallest element is at a[j];
    Interchange a[i] and a[j];
    }

- Example: Selection sort pseudo code program

```
1 Algorithm SelectionSort (a,n)
2  // sort the array a[1:n] into increasing order.
3  {
4    for i:=1 to n do
5     {
6       j:=i;
7       for k:= i+1 to n do
8          if (a[k]<a[j]) then j:=k;
9       t:=a[i]; a[i]:=a[j]; a[j]:=t;
10    }
11  }
```

- Recursive Algorithms:
  - A function that is defined in terms of itself is called recursive function.
  - An algorithm can be direct recursive or indirect recursive.
  - Any algorithm that can be written using assignment, if-then-else and while statement can also be written using assignment, if-then-else and recursion. Sometimes given problem is itself recursively defined.
  - Example: Towers of Hanoi:
    - Three diamond towers are there out of which one diamond tower has 64 golden disks arranged in decreasing size from bottom to top. Disks are required to move from one tower to another using the third tower. Only one disk can be moved at a time and at no time can a disk be on top of a smaller disk.

  - Towers of Hanoi using recursion:

```
1 Algorithm TowersOfHanoi(n,x,y,z)
2 {
3   if (n>=1) then
4   {
5     TowersOfHanoi(n-1,x,z,y);
6     write ("move top disk from tower", x, "to top of
      tower", y);
7      TowersOfHanoi(n-1,z,y,x)
8    }
9 }
```

- Another example: Permutation Generator:
  - Given a set of n>=1 elements, problem is to print all possible permutations. If set is {a, b, c} then set of permutations is {(a,b,c), (a,c,b), (b,a,c), (b,c,a), (c,a,b), (c,b,a)}. For given n numbers there are n! different permutations.

```
1   Algorithm Perm(a,k,n)
2   {
3     if (k=n) then write (a[1:n]);
4     else
5         for i:=k to n do
6         {
7           t:=a[k];a[k]:=a[i];a[i]:=t;
8           Perm(a,k+1,n);
9           t:=a[k];a[k]:=a[i];a[i]:=t;
10        }
11  }
```

# Performance Analysis

- Space Complexity of an algorithm is the amount of memory it needs to run to completion.
- Time Complexity is the amount of computer time it needs to run to completion.
- Performance Evaluation is divided into two major phases:
  - A priori estimates (performance analysis)
  - A posteriori testing (performance measurement)

- Space Complexity:
  - Space needed is sum of following components:
    - Fixed part that is independent of characteristics of inputs and outputs (instance characteristics). For eg. Space for code, space for constants.
    - Variable part consists of space needed by variables whose size is dependent on particular problem instance.
    - S(P) = c+Sp where c is constant and Sp depends on instance characteristics. Focus is basically on Sp (instance characteristics). It is also important to determine which instance characteristics to use to measure space requirements.
    - Example:

```
1. Algorithm abc(a,b,c)
2. {
3.     return a+b+b*c+(a+b-c)/(a+b)+4.0;
4. }
```

Space needed by this algorithm is independent of instance characteristics so Sp=0

- Example 2:

```
1.  Algorithm Sum(a,n)
2.  {
3.     s:=0.0;
4.     for i:=1 to n do
5.        s:=s+a[i];
6.     return s;
7.  }
```

This problem instance is characterized by n. n, s and i are using one word. Array a is using >= n words. So $S_{sum}(n) >= (n+3)$

- Example 3:

```
1. Algorithm Rsum(a,n)
2. {
3.    if (n<=0) then return 0.0;
4.    else return Rsum(a,n-1)+a[n];
5. }
```

This problem instance is characterized by n. Recursion stack space includes space for formal parameters, local variables and return address. Each call requires atleast three words (for n, return address and pointer to a[]) As depth of recursion is n so space needed is >= 3n

- Time Complexity:
  - Time T(P) taken by a program P is sum of compile time and run time. As compile time does not depend on instance characteristics so focus is on run time denoted by t(p).
  - One way to calculate run time is to add additions, multiplications, compares etc.
    - $Tp(n) = c_aADD(n)+c_sSUB(n)+c_mMUL(n)$……
    - It is impossible to obtain such an exact formula.
  - Another way is to count number of program steps (meaningful segment of a program that has an execution time).
  - Return a+b+b*c+(a+b-c)/(a+b)+4.0 can be regarded as one step.
  - Number of steps any program statement is assigned depends on kind of statement. A comment can be assigned as zero steps, assignment statement as one step and so on.

- One way to determine number of steps is to introduce count (initialized to zero) in the program.
  - Example:

```
1   Algorithm Sum(a,n)
2   {
3     s:=0.0;
4     count:=count +1;
5     for i:=1 to n do
6     {
7        count:=count+1;//For for i
8        s:=s+a[i]; count := count+1// For assignment
9     }
10    count:=count+1 //For last time of for
11    count:=count+1;//For return
12    return s;
13  }
```

– Simplified Algorithm

```
1   Algorithm Sum(a,n)
2   {
3      for i:=1 to n do count:=count+2;
4      count:=count+3;
5   }
```

In the for loop value of count will increase by 2n. So each invocation of Sum executes total of 2n+3 steps.

–Example 2:

```
1   Algorithm Rsum(a,n)
2   {
3     count:=count+1; //For if condition
4     if (n<=0), then
5     {
6        count:=count+1; //For return
7        return 0.0;
8     }
9     else
10    {
11       count:=count+1; //for addition, function invocation &
                         //return
12       return Rsum(a,n-1)+a[n];
13    }
14  }
```

Let $t_{RSum}(n)$ be the increase in value of count when algorithm terminates. $t_{RSum}(0) = 2$. When n >0, count increases by 2 plus increase results from invocation of RSum from within else ie. $t_{RSum}(n-1)$.

$$t_{RSum(n)} = \begin{cases} 2 & \text{If } n=0 \\ 2+t_{RSum}(n-1) & \text{if } n>0 \end{cases}$$

(These recursive formulas are called recurrence relations)

– These relations can be solved by repeated substitutions

– $t_{RSum}(n) = 2 + t_{RSum}(n-1)$

$= 2 + 2 + t_{RSum}(n-2)$

$= 2(2) + t_{RSum}(n-2)$

$= n(2) + t_{RSum}(0)$

$= 2n+2$

This step count tells that run time for a program changes with changes in instance characteristics. If n is doubled, run time doubles ie run time grows linearly in n.

– Example 3:

```
1   Algorithm Add(a,b,c,m,n)
2   {
3         for i:=1 to m do
4            for j:=1 to n do
5               c[i,j]:=a[i,j]+b[i,j];
6    }
```

– Algorithm with counting

```
1   Algorithm Add(a,b,c,m,n)
2   {
3      for i:= 1 to m do
4      {
5         count:=count+1; //For for i
6         for j:=1 to n do
7         {
8            count:=count +1; //For for j
9            c[i,j]:=a[i,j]+b[i,j];
10           count:=count+1; //For assignment
11        }
12        count:=count+1; //for last time of for j
13     }
14     count:=count+1; //For last time of for i
15  }
```

– Simplified Algorithm

```
1   Algorithm Add(a,b,c,m,n)
2   {
3     for i:=1 to m do
4     {
5        count:=count+2;
6        for j:=1 to n do
7           count:=count+2;
8      }
9      count:=count+1;
10  }
```

Line 7 is executed mn times. Line 5 is executed m times and line 9 is executed 1 time. When algorithm terminates value of count will be 2mn+2m+1.

- Second method to determine step count is to build a table.
- Example:

| Statement | s/e | Frequency | Total steps |
|---|---|---|---|
| 1. Algorithm Sum(a,n)<br>2. {<br>3. s:=0.0;<br>4. for i:=1 to n do<br>5. s:=s+a[i];<br>6. return s;<br>7. } | 0<br>0<br>1<br>1<br>1<br>1<br>0 | -<br>-<br>1<br>n+1<br>n<br>1<br>- | 0<br>0<br>1<br>n+1<br>n<br>1<br>0 |
| Total | | | 2n+3 |

- Example 2:

| Statement | s/e | Frequency | | Total steps | |
|---|---|---|---|---|---|
| | | n=0 | n>0 | n=0 | n>0 |
| 1. Algorithm Rsum(a,n) | 0 | - | - | 0 | 0 |
| 2. { | 0 | - | - | 0 | 0 |
| 3. if (n<=0) then | 1 | 1 | 1 | 1 | 1 |
| 4. return 0.0; | 1 | 1 | 0 | 1 | 0 |
| 4. else return Rsum(a,n-1+a[n]; | 1+x | 0 | 1 | 0 | 1+x |
| 5. } | 0 | - | - | 0 | 0 |
| Total | | | | 2 | 2+x |

$x=t_{RSum}(n-1)$

- Example 3:

| Statement | s/e | Frequency | Total steps |
|---|---|---|---|
| 1 Algorithm Add(a,b,c,m,n) | 0 | - | 0 |
| 2 { | 0 | - | 0 |
| 3 for i:=1 to m do | 1 | m+1 | m+1 |
| 4 for j:=1 to n do | 1 | m(n+1) | mn+m |
| 5 c[i,j]:=a[i,j]+b[i,j]; | 1 | mn | mn |
| 6 } | 0 | - | 0 |
| Total | | | 2mn+2m+1 |

- After obtaining sufficient experience step count can be calculated without making any table
- Example: To obtain nth fibonacci term ($f_n=f_{n-1}+f_{n-2}$, n>=2:

```
1.  Algorithm Fibonacci(n)
2.  //Compute nth fibonacci number.
3.  {
4.    if (n<=1) then
5.      write(n);
6.    else
7.    {
8.       fnm2:=0; fnm1:=1;
9.       for i:=2 to n do
10.      {
11.        fn:=fnm1+fnm2;
12.        fnm2:=fnm1; fnm1:=fn;
13.      }
14.      write(fn);
15.   }
16.  }
```

Two cases need to be considered:
Case1: When n=0 or 1
Lines 4 and 5 get executed once each. So total step count is 2.
Case2: When n>1
Lines 4, 8 and 14 are executed once. Line 9 gets executed n times and lines 11 and 12 get executed n-1 times. Lines 8 and 12 have s/e of 2. So total steps for n>1 is 4n+1

- Summarized Time Complexity:
  - Time Complexity is not dependent solely on number of inputs or outputs. An algorithm can terminate in one step also.
  - Three types of step counts can be defined:
    - Best case: Minimum no. of steps executed for the given parameters.
    - Worst case: Maximum no. of steps executed for the given parameters.
    - Average case: Average no. of steps executed for the given parameters.
  - Aim is to be able to compare time complexities of two algorithms that compute same function and also to predict the growth in run time as instance characteristics change.
  - Determining exact step count is exceedingly difficult and also not very worthwhile as notion of a step is itself inexact.
  - It is adequate to make a statement like $t(n,m)=c_1n+c_2m$ or $c_1n^2<=t(n)<=c_2n^2$

- If we have two algorithms with complexity of $c_1n^2+c_2n$ and $c_3n$ respectively then the one with complexity $c_3n$ will be faster than the one with complexity $c_1n^2+c_2n$ for sufficiently large value of n. For small value of n either algorithm could be faster. For eg. $c_1=1$, $c_2=2$ and $c_3=100$ then $c_1n^2+c_2n <= c_3n$ for $n<=98$ and $c_1n^2+c_2n > c_3n$ for $n>98$.
- Irrespective of the values of $c_1$, $c_2$ and $c_3$ there will be an n beyond which the algorithm with complexity $c_3n$ will be faster then the other algorithm. This value of n is called break-even point. If break even point is zero then the algorithm with complexity $c_3n$ will always be faster. Exact break-even point can not be determined analytically.

# Asymptotic Notation

- Most often people are interested in the rate of growth of the time and space required to solve larger and larger instances of a problem.
- Time needed by an algorithm expressed as a function of the size of a problem is called time complexity.
- The limiting behavior of the complexity as size increases in called the asymptotic time complexity.
- It is assumed that functions f and g are nonnegative functions.
- Big "oh"(O): $f(n)=O(g(n))$ iff there exist positive constants c and $n_0$ such that $f(n)<=c*g(n)$ for all n, $n>=n_0$.
  - O(1) is used for computing time that is constant. O(n) is called linear. $O(n^2)$ is called quadratic, $O(n^3)$ is called cubic and $O(2^n)$ is called exponential.
  - Algorithm that takes time O(log n) is faster, for sufficiently large n, than the algorithm that takes time O(n). Similarly O(nlogn) is better than $O(n^2)$ but not as good as O(n)
  - g(n) is basically an upper bound on the value of f(n) for all n, $n>=n_0$. g(n) should be as small a function of n as one can come up with for which f(n)=O(g(n)).
  - For eg. For $3n+3=O(n)$, $3n+3=O(n^2)$ is also correct but we never use it.

- Example:
  - f(n)=3n+2=O(n) if we take c=4, $n_0$=2.
  - 3n+2 ≠ O(1) as 3n+2 is not less than equal to c for any value of c and n.
  - $10n^2+4n+2 \neq O(n)$
- Theorem: If $f(n)=a_m n^m+.....a_1 n+a_0$, then $f(n)=O(n^m)$

- Omege (Ω): f(n) = Ω(g(n)) iff there exist positive constants c and $n_0$ such that f(n)>=c*g(n) for all n, n>=$n_0$.
  - g(n) is only a lower bound on f(n) and it should be as large a function of n as possible.
  - For eg. 3n+3= Ω(n). 3n+3= Ω(1) is also correct but we never use it.
  - Example:
    - 3n+2= Ω(n) as 3n+2>=3n for n>=1.
    - $10n^2+4n+2 = \Omega(n^2)$ as $10n^2+4n+2 >= n^2$ for n>=1.
  - Theorem: If $f(n)=a_m n^m+.....a_1 n+a_0$, and $a_m>0$ then $f(n)= \Omega(n^m)$

- Theta(Θ): f(n)= Θ(g(n)) iff there exist positive constants $c_1$, $c_2$ and $n_0$ such that $c_1 g(n)<=f(n)<=c_2 g(n)$ for all n, n>=$n_0$.
  - This notation is more precise that both the big oh and omega notations. g(n) is both an upper and lower bound on f(n).
  - Theorem: If $f(n)=a_m n^m+.....a_1 n+a_0$, and $a_m>0$ then $f(n)= \Theta(n^m)$
- In accordance with practice, the coefficients in all g(n)'s are 1.
- Little "oh": f(n)=o(g(n)) iff $\lim \frac{f(n)}{n->\infty g(n)} = 0$
  - For eg. $3n+2=o(n^2)$ as lim n-> ∞ $3n+2/n^2=0$. 3n+2=o(nlogn). But 3n+2 ≠ o(n)
- Little omega: f(n) = ω(g(n)) iff $\lim \frac{g(n)}{n->\infty f(n)} = 0$
- For algorithm Sum tsum(n)=2n+3, so tsum(n)= Θ(n)

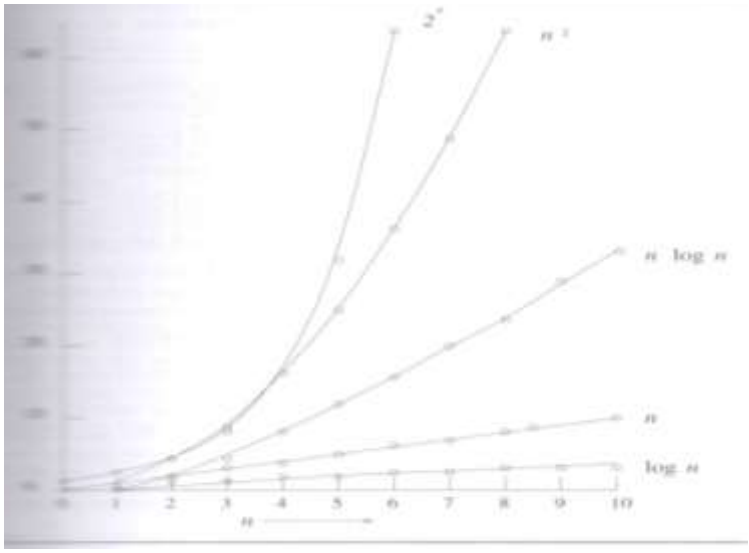
| Statement | s/e | Frequency | Total steps |
|---|---|---|---|
| 1. Algorithm Sum(a,n)<br>2. {<br>3. s:=0.0;<br>4. for i:=1 to n do<br>5. s:=s+a[i];<br>6. return s;<br>7. } | 0<br>0<br>1<br>1<br>1<br>1<br>0 | -<br>-<br>1<br>n+1<br>n<br>1<br>- | Θ(0)<br>Θ(0)<br>Θ(1)<br>Θ(n)<br>Θ(n)<br>Θ(1)<br>Θ(0) |
| Total | | | Θ(n) |

# Practical Complexities

- Time complexity is generally some function of instance characteristics. Complexity function can be used to compare two algorithms P and Q that perform the same task.
- An algorithm P with complexity Θ(n) is better than an algorithm Q with complexity Θ($n^2$) for sufficiently large n. As P is bounded from above by cn for some constant c and n>=n1 and Q is bounded from below by $dn^2$ for some constant d and n>=n2. As cn<=$dn^2$ for n>=c/d so P is faster than Q whenever n>= max{n1,n2,c/d}
- Following table shows how a function grows with n.

| **log n** | **n** | **nlogn** | **$n^2$** | **$n^3$** | **$2^n$** |
|---|---|---|---|---|---|
| 0 | 1 | 0 | 1 | 1 | 2 |
| 1 | 2 | 2 | 4 | 8 | 4 |
| 2 | 4 | 8 | 16 | 64 | 16 |
| 3 | 8 | 24 | 64 | 512 | 256 |
| 4 | 16 | 64 | 256 | 4096 | 65536 |
| 5 | 32 | 160 | 1024 | 32768 | 4294967296 |

## Plot of function values




# DIVIDE AND CONQUER

# Divide and Conquer

- Splitting the inputs into k distinct subsets yielding k sub problems. These sub problems are solved and then sub solutions are combined to form the final solution.
- Sub problems are often of the same type as the original problem. This can naturally be expressed by a recursive algorithm.
- Example: Detecting a counterfeit coin: There is a bag having 16 coins out of which one may be counterfeit. A counterfeit coin is lighter than genuine one. Divide and conquer strategy can solve this problem in comparatively less number of comparisons (4 instead of 8).
- Control Abstraction of an algorithm shows flow of control. Primary operations are specified by other procedures whose precise meanings are left undefined.

33

– Algorithm DAndC(P)

```
{ if Small(P) then return S(P);
  else
  { divide P into smaller instances P1,P2,…Pk, k>=1;
    Apply DAndC to each of these subproblems;
    Return Combine(DAndC(p1),DAndC(P2)….DAndC(Pk));
  }
}
```

– Small(P) is a boolean valued function that determines whether input size is small enough that the answer can be computed without splitting.

– Combine is a function that combines the sub solutions.

– The computing time of DAndC is described by the recurrence relation:

- $T(n)= \begin{cases} g(n) & n \text{ small} \\ T(n1)+T(n2)+\ldots\ldots T(nk) + f(n) & \text{otherwise} \end{cases}$
- g(n) is time to compute answer directly for small inputs.
- f(n) is the time for dividing P and combining the solutions to subproblems.

34

- Complexity of many divide and conquer algorithms is given by recurrences of the form
  - $T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$
  - a and b are known constants
  - Assume that T(1) is known and n is a power of b ie. $n = b^k$.
  - Such recurrence relations can be solved using substitution method.
- Example: Let a =2 and b=2. Let T(1)=2 and f(n)=n then
  - $T(n) = 2T(n/2)+n$
    $= 2[2T(n/4)+n/2] + n$
    $= 4T(n/4) + n + n$
    $= 4T(n/4)+2n$
    $= 4[2T(n/8)+n/4]+2n$
    $= 8T(n/8)+3n$
    $= \ldots$
    $= 2^kT(n/2^k)+kn$
  - In general maximum value of k is $\log_2 n$ and $2^{\log_2 n}$ is n.
  - So $T(n) = nT(1)+n \log_2 n$ ie. $n\log_2 n + 2n$.

35

# Binary Search

- Let $a_i$, 1<=i<=n, be a list of elements sorted in increasing order and a given element x is to be found in the list. j is supposed to have the position of x in the list when $a_j$=x otherwise j is set to zero.
- Let $P=(n,a_i,\ldots a_l,x)$ is an instance of the search problem. If Small(P) is true ie. n=1, then S(P) will return i if $x=a_i$ or zero if $x \neq a_i$. So $g(1) = \Theta(1)$. If n >0, then an index q between i and l can be picked such that if $a_q = x$ then j = q, if $x < a_q$ then P becomes $P=(q - i, a_i,\ldots a_{q-1}, x)$, if $x > a_q$ then P becomes $P = (l - q, a_{q+1},\ldots a_l, x)$. If q is chosen such that $a_q$ is the middle element ie. q = floor((n+1)/2), then the algorithm is binary search algorithm. Combining of solutions is not required in this algorithm.

36

- Recursive Binary Search Algorithm
- Algorithm BinSrch(a,i,l,x)

```
{   if(l=i) then
    {       if (x = a[i]) then return i;
            else return 0;
    }
    else
    {       mid:= ⌊(i+1)/2⌋;
            if(x=a[mid] then return mid;
            else
            {       if (x<a[mid]) then
                            return BinSrch(a,i,mid-1,x);
                    else
                            return BinSrch(a,mid+1,l,x);
            }
    }
}
```

37

- Iterative Binary Search Algorithm
- Algorithm BinSrch(a,n,x)

```
{
    low:=1; high:=n;
    while (low <= high) do
    {       mid := ⌊(low+high)/2⌋;
            if (x < a[mid] then high:=mid+1;
            else if ( x > a[mid]) then low:=mid+1;
            else return mid;
    }
    return 0;
}
```

38

- Testing Binary Search Algorithm: To test all successful searches x must take on n values in a. To test all unsuccessful searches, x need only take on n+1 different values. So, complexity of testing Binary Search algorithm is 2n+1 for each n.
- Space required for this algorithm includes space for n elements of array. Space for low, high, mid and x ie. Total n+4 locations are required.
- Complexity of Binary Search Algorithm:
  - Comparisons in Binary Search algorithm can be described in the form of a binary decision tree. For eg. if n=14:
  - If x is present, the algorithm will end at one of the circular nodes (internal nodes).
  - If x is not present, algorithm will terminate at one of the square nodes (external nodes).
  - Theorem: If n is in the range $[2^{k-1},2^k)$, then BinSearch makes at most k element comparisons for a successful search and either k-1 or k comparisons for an unsuccessful search ie. time complexity for a successful search is O(log n) and for an unsuccessful search is Θ(log n)
- For successful searches: Best- Θ(1), Worst- Θ(log n), Average- Θ(log n)
- For unsuccessful searches: Best, Worst, Average- Θ(log n)

39

# Merge Sort

- Its worst case complexity is O(n log n)
- n elements from a[1] to a[n] are split into two sublists a[1]……. a[$\lfloor n/2 \rfloor$] and a[$\lfloor n/2 \rfloor$ +1].....a[n]. Each sublist is individually sorted and then merged to produce single sorted list.
- Algorithm MergeSort(low, high)

```
{     if (low < high) then
      {       mid := ⌊(low+high)/2⌋;
              MergeSort(low, mid);
              MergeSort(mid+1,high);
              Merge(low, mid, high);
      }
}
```

40

- Algorithm Merge(low, mid, high)

```
{       h:= low; i:= low, j:= mid+1;
        while((h<=mid) and (j<=high)) do
        {       if(a[h]<=a[j]) then
                {       b[i]:=a[h];  h:=h+1;
                }
                else
                {       b[i]:=a[j];   j:=j+1;
                }
                i:=i+1;
        }
        if(h>mid) then
        {       for k:=j to high do
                {       b[i]:=a[k];  i:=i+1;
                }
        }
        else
        {       for k:=h to mid do
                {       b[i]:=a[k];  i:=i+1;
                }
        }
        for k:=low to high do a[k]:=b[k];
}
```

41

- Example:
  - {179, 254, 285, 310, 351, 423}
- Computing Time of Merge Sort is described by recurrence relation:
  - $T(n) = \begin{cases} a & n = 1, a \text{ is a constant} \\ 2T(n/2) + cn & n > 1, c \text{ is a constant} \end{cases}$
  - When n is a power or 2, $n= 2^k$, then ‘

$$\begin{aligned} T(n) &= 2(2T(n/4)+cn/2)+cn \\ &= 4T(n/4)+2cn \\ &= \ldots\ldots \\ &= 2^kT(1)+kcn \\ &= an + c \text{ nlog } n \end{aligned}$$

  - If $2^k<n<=2^{k+1}$, then $T(n)<=T(2^{k+1})$ , so $T(n)= O(n\log n)$

42

# Quick Sort

- Division into two subarrays is made so that sorted subarrays do not need to be merged later.
- The rearrangement of elements by picking some element of a[] and then reordering the other elements so that all elements appearing before that element are less and all the elements appearing after that element are greater than that element. This rearranging is called partitioning.
- For eg;
  - given list is: 65 70 75 80 85 60 55 50 45

43

```
Algorithm Partition(a,m,p)
{       v:=a[m]; i:=m; j:=p;
        repeat
        {           repeat
                    {           i:=i+1;
                    }until (a[i]>=v);
                    repeat
                    {           j:=j-1;
                    }until(a[j]<=v);
                    if(i<j) then Interchange(a,i,j);
        }until(i>=j);
        a[m]:=a[j]; a[j]:=v; return j;
}
Algorithm Interchange(a,i,j)
{       p:=a[i]; a[i]:=a[j];a[j]:=p;
}
```

44

- Algorithm QuickSort(p,q)

```
{   if(p<q) then
    {       j:=partition(a, p, q+1);
            QuickSort(p,j-1);
            QuickSort(j+1,q);
    }
}
```

- Worst and Average Case Time Complexity of Quick sort is $O(n^2)$ and O(n logn) respectively.
- Space complexity of Quick sort is O(n).

45

# Selection Problem.

- In this problem, n elements are given and it is required to determine kth smallest element. If partitioning element is positioned at a[j] then j-1 elements are <= a[j] and n-j elements are >= a[j]. If k<j then kth smallest element is in a[1:j-1]; if k=j, then a[j] is the kth smallest element; if k>j, then kth smallest element is the k-jth smallest element in a[j+1:n]

46

- Algorithm Select(a,n,k)

```
{   low:=1; up:=n+1;
    a[n+1]:= infinity;
    repeat
    {       j:=partition(a,low,up);
            if (k=j) then return  a[j];
            else if (k<j) then
                    up:=j;
            else
                    low:=j+1;
    }until(false);
}
```

Example.

47

- Analysis of Selection Algorithm:
  - Partition requires O(n) time
  - On each successive call to partition either m increases by at least one or j decreases by at least one. So atmost n calls to partition can be made.
  - So worst case complexity of Selection algorithm is $O(n^2)$. Worst case is when the largest element is to be found and the list is sorted.
  - Best case is when the first element in the unsorted list is the kth smallest element. Best case complexity is O(n).
  - The average computing time of Selection algorithm is O(n).

48

# MATRIX MULTIPLICATION using DIVIDE & CONQUER

- The product of two n x n matrices A, B is given by
  $C(i, j) = \sum_{1 \le k \le n} A(i, k)\, B(k, j)$
  for all i, j $1 \le i \le n$, $1 \le j \le n$
- The matrix C has $n^2$ elements.
- To compute C(i, j) we need n multiplications, so the time is $O(n^3)$.

49

Divide and Conquer strategy to compute the product

- Assume n is a power of 2 i.e. $n = 2^k$ for some $k \ge 0$.
- If $n \ne 2^k$, we can add rows and columns of zeros to both A and B to make $n=2^k$
- If n > 2, partition A and B into square sub matrices each having dimensions n/2 x n/2.
- Each sub matrix is partitioned into sub matrices and so on.
- These matrix products can be recursively computed by the same algorithm used for n x n case.
- The algorithm will continue until n becomes suitably small (n=2).
- Then the product is computed directly as

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

50

where

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$
$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$
$$C_{21} = A_{21}B_{11} + A_{21}B_{21}$$
$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

51

Example :

Let A = $\begin{bmatrix} 1 & 2 & 1 & 1 \\ 2 & 1 & 1 & 1 \\ 1 & 1 & 1 & 2 \\ 1 & 0 & 0 & 1 \end{bmatrix}$ B = $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 2 & 1 \\ 1 & 2 & 1 & 1 \end{bmatrix}$

AB = $\begin{bmatrix} 3 & 5 & 5 & 4 \\ 4 & 4 & 4 & 3 \\ 4 & 6 & 5 & 4 \\ 2 & 2 & 1 & 1 \end{bmatrix}$

A = $\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ B = $\begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$

52

where

$$A_{11} = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \quad A_{12} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad A_{21} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \quad A_{22} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

$$B_{11} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad B_{12} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \quad B_{21} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} \quad B_{22} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$$

53

$$C_{11} = A_{11}\,B_{11} + A_{12}\,B_{21} = \begin{bmatrix} 3 & 5 \\ 4 & 4 \end{bmatrix}$$

$$C_{12} = A_{11}\,B_{12} + A_{12}\,B_{22} = \begin{bmatrix} 5 & 4 \\ 4 & 3 \end{bmatrix}$$

$$C_{21} = A_{11}\,B_{11} + A_{12}\,B_{21} = \begin{bmatrix} 4 & 6 \\ 2 & 2 \end{bmatrix}$$

$$C_{22} = A_{11}\,B_{12} + A_{12}\,B_{22} = \begin{bmatrix} 5 & 4 \\ 1 & 1 \end{bmatrix}$$

54

- Analysis of Matrix Multiplication using DANC
  - The computing time T(n) to multiply two n x n matrices is

$$T(n) = \begin{cases} b & n \leq 2 \\ 8T(n/2) + cn^2 & n > 2 \end{cases}$$

  - Two n x n matrices require 8 multiplications of n/2 x n/2 matrices and four additions.
  - Time Complexity of Matrix Multiplication using DANC is $O(n^3)$

- Volker strassen's Matrix Multiplication
  - Matrix Multiplication is more expensive than Matrix Addition so equations for $C_{ij}$ can be reformulated so as to have fewer multiplications.
  - Volker Strassen has discovered a way to compute $C_{ij}$ using only 7 multiplications and 18 additions or subtractions

55

$P = (A_{11} + A_{22}) (B_{11} + B_{22})$
$Q = (A_{21} + A_{22}) (B_{11})$
$R = A_{11} (B_{12} – B_{22})$
$S = A_{22} (B_{21} – B_{11})$
$T = (A_{11} + A_{12}) B_{22}$
$U = (A_{21} – A_{11}) (B_{11} + B_{12})$
$V = (A_{12} – A_{22}) (B_{21} + B_{22})$
$C_{11} = P+S-T+V$
$C_{12} = R+T$
$C_{21} = Q+S$
$C_{22} = P+R-Q+U$

- Here $A_{11}$ , $A_{12}$ , ….. $B_{11}$ , $B_{12}$ , …. are matrices of order n/2.
- For strassen's matrix multiplication computing time T(n) is given by:

$$T(n) = \begin{cases} b & n<=2 \\ 7T(n/2) + an^2 & n>2 \end{cases}$$

- Time complexity of this algorithm is $O(n^{2.81})$

56

# GREEDY METHOD

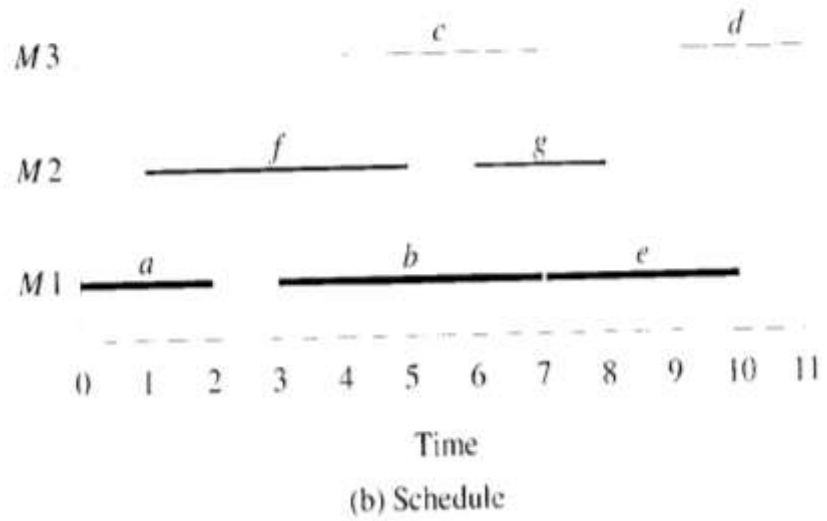
## The Greedy Method

- In some type of problems, we have n inputs and we are required to obtain a subset that satisfies some constraints.
- Any subset that satisfies these constraints is called a feasible solution.
- A feasible solution that either maximizes or minimizes a given objective function is called optimal solution.
- Greedy method suggests that an algorithm can be devised that works in stages considering one input at a time. At each stage, a decision is made whether a particular input is in an optimal solution. This is done by including the next input into the partially constructed optimal solution and checking whether the inclusion results in an infeasible solution? If yes, that input is not added otherwise it is included.
- Selection procedure for selecting next input is based on some optimization measure.
- This version of greedy technique is called ‘subset paradigm’.
- When we make decisions by considering the inputs in some order, that greedy technique is called ‘ordering paradigm’.

- Example 1:
  - Change Making
    - For giving change to someone, suppose the cashier wants to return fewest number of notes or coins.
    - He can work on the greedy criterion: At each stage increase the total amount of change constructed by as much as possible.
    - For eg. for returning Rs. 67, he will take one note of Rs. 50, one of Rs. 10, one of Rs. 5 and one note of Rs. 2.
  - Machine Scheduling
    - n tasks are given and an infinite supply of machines is there.
    - Each task has a start time $s_i$ and a finish time $f_i$, $s_i<f_i$
    - Two tasks overlap if their processing intervals overlap at a point other than the interval start or end.
    - A feasible task-to-machine assignment is the one in which no machine is assigned two overlapping tasks.
    - An optimal assignment is a feasible assignment that utilizes the fewest number of machines.

| task | a | b | c | d | e | f | g |
|---|---|---|---|---|---|---|---|
| start | 0 | 3 | 4 | 9 | 7 | 1 | 6 |
| finish | 2 | 7 | 7 | 11 | 10 | 5 | 8 |

- For the above problem, one solution is to use 7 machines for 7 tasks. This solution is feasible but not optimal.
- Using greedy strategy, we will assign tasks in stages. Call a machine OLD if some task has already been assigned to it, other the machine is NEW.
- The greedy criterion is: if an old machine becomes available by the start time of the task to be assigned, assign the task to this machine, otherwise assign it to a NEW machine.
- We are considering tasks in the increasing order of task start time.



(b) Schedule

# Knapsack Problem

- n objects and a knapsack is given. Object i has a weight wi and knapsack has capacity m. If a fraction xi, 0<=xi<=1 of object i is placed into the knapsack, then the profit of $p_i x_i$ is earned.
- Objective is to fill the knapsack such that the total profit earned is maximized ie the total weight of all chosen objects should be atmost m.

  maximize $\sum_{1<=i<=n} p_i x_i$

  subject to $\sum_{1<=i<=n} w_i x_i <= m$

  and $0<=x_i<=1,\ 1<=i<=n$
- Feasible solution is any set $x_1, x_2, \ldots x_n$ that satisfies last two equations. Optimal solution is a feasible solution that maximizes first equation.

- Example:
  - n=3, m=20, $(p_1,p_2,p_3)=(25,24,15)$, $(w_1,w_2,w_3) = (18,15,10)$
  - No. of feasible solution:

| $(x_1, x_2, x_3)$ | $\sum w_i x_i$ | $\sum p_i x_i$ |
|---|---|---|
| (1/2,1/3,1/4) | 16.5 | 24.25 |
| (1,2/15,0) | 20 | 28.2 |
| (0,2/3,1) | 20 | 31 |
| (0,1,1/2) | 20 | 31.5 |

  - Lemma: If the sum of all the weights is less or equal to m then $x_i=1$ for i=1 to n is an optimal solution.
  - Lemma: All optimal solutions will fill the knapsack exactly. This is true as we can always increase the contribution of some object by a fractional amount until the total weight is exactly m.
  - Knapsack problem is an example of subset paradigm problem.

- Greedy Strategies:
  - Pick the objects in the order of decreasing profit. If the object under consideration does not fit, then a fraction of it is included.
    - This strategy does not give optimal solution. For eg. If 2 units space is left and we have two objects to choose from ($p_i=4$, $w_i=4$) and ($p_i=3$, $w_i=2$). Then it is better to choose 2nd object.
  - Consider objects in order of increasing weights ie use the capacity as slowly as possible.
    - This strategy again does not give optimal solution. For eg. If 2 units space is left and we have two objects to chose from (pi=4, wi=2) and (pi=10, wi=4). Then it is better to choose 2nd object.
  - Consider objects in order of decreasing profit per unit weight ratio ie. $p_i/w_i$.
  - Disregarding the time to initially sort the objects, each of the three strategies require only O(n) time.

- Algorithm GreedyKnapsack(m,n)

```
{   for i:= 1 to n do x[i]:=0.0;
    U:=m;
    for i:=1 to n do
    {       if (w[i]>U) then break;
            x[i]:=1.0;
            U:=U-w[i];
    }
    if(i<=n) then x[i]:=U/w[i];
}
```

- Theorem: If $p_1/w_1>=p_2/w_2>=\ldots\ldots>=p_n/w_n$, then GreedyKnapsack generates optimal solution.
- Proof:
  - Let $x=\{x_1,x_2\ldots.,x_n)$ be the solution generated by GreedyKnapsack.
  - If all $x_i$ are equal to one then solution is optimal. Otherwise
  - Let j be the least index such that $x_j \neq 1$. So $x_i = 1$ for $1<=i<j$ and $x_i=0$ for $j<i<=n$ and $0<=x_j<1$.
  - Let $y=(y_1,y_2,\ldots,y_n)$ be the optimal solution and $\sum w_iy_i=m$.
  - Let k be the least index such that $y_k \neq x_k$ .
  - Now it is clear that $y_k<x_k$:
    - There are three possibilities: k<j, k=j and k>j.
    - If k<j then $x_k=1$. But $y_k \neq x_k$ so $y_k<x_k$.
    - If k=j, then since $\sum w_ix_i=m$ and $y_i=x_i$ for $1<=i<j$, so either $y_k<x_k$ or $\sum w_iy_i>m$
    - If k>j then $\sum w_iy_i>m$ which is not possible.
  - Suppose we increase $y_k$ to $x_k$ and decrease as many of $(y_{k+1},\ldots.y_n)$ as necessary so that the total capacity used is still m. This gives a new solution $z=(z_1,z_2,\ldots.z_n)$ with $z_i=x_i$, $1<=i<=k$ and $\sum (k<i<=n)w_i(y_i-z_i) = wk(z_k-y_k)$. Then

$$\begin{aligned}\sum_{1\le i\le n} p_i z_i &= \sum_{1\le i\le n} p_i y_i + (z_k - y_k)w_k\frac{p_k}{w_k} - \sum_{k<i\le n}(y_i - z_i)w_i\frac{p_i}{w_i} \\ &\ge \sum_{1\le i\le n} p_i y_i + \left[(z_k - y_k)w_k - \sum_{k<i\le n}(y_i - z_i)w_i\right]\frac{p_k}{w_k} \\ &= \sum_{1\le i\le n} p_i y_i\end{aligned}$$

# Job Sequencing with Deadlines

- n jobs are given. With each job i its profit and deadline are associated (di and pi). pi is earned if job is completed by its deadline. Only one machine is available and each job requires one unit of time to execute.
- A feasible solution is a subset J of jobs such that each job in J can be completed by its deadline. An optimal solution is a feasible solution with maximum value of ∑ (i∈J)pi.
- It's a subset paradigm problem.
- Example: n=4, (p1,p2,p3,p4)=(100,10,15,27) and (d1,d2,d3,d4)=(2,1,2,1).
  - Feasible solutions are:

| | feasible solution | processing sequence | value |
|---|---|---|---|
| 1. | (1, 2) | 2, 1 | 110 |
| 2. | (1, 3) | 1, 3 or 3, 1 | 115 |
| 3. | (1, 4) | 4, 1 | 127 |
| 4. | (2, 3) | 2, 3 | 25 |
| 5. | (3, 4) | 4, 3 | 42 |
| 6. | (1) | 1 | 100 |
| 7. | (2) | 2 | 10 |
| 8. | (3) | 3 | 15 |
| 9. | (4) | 4 | 27 |

- It is required to formulate an optimization criteria for determining how the next job is chosen.
- We can chose the objective function ∑ (i∈J)pi as the optimization measure. So jobs should be considered in the decreasing order of the pi's.
- In the beginning J = Ø, first job with highest profit is selected and it is checked whether that job can be completed within its deadline. If yes it is added otherwise it is rejected. Then next job with highest profit is selected and the process continues.
- Jobs in J can be executed in any order. So to check the feasibility of jobs in J, all the possible permutations of the jobs in J should be tried out. If |J|=i, this requires i! Permutations. Another way is to check only one permutation in which jobs are ordered in increasing order of deadlines.

- Theorem: Let J be a set of k jobs and σ = i1, i2,….ik a permutation of jobs in J such that di1<=di2<=……dik. Then J is a feasible solution iff jobs in J can be processed in the order σ without violating any deadline.
- Proof: We have to show that if J is feasible then σ represents a possible permutation.
  - If J is feasible, then there exists σ' = $r_1$,$r_2$,…..$r_k$ such that $dr_q$>=q, 1<=q<=k.
  - Assume σ' ≠ σ, then let a be the least index such that $r_a \neq i_a$. Let $r_b = i_a$. Clearly b>a.
  - In σ', we can interchange $r_a$ and $r_b$. Since $dr_a$>=$dr_b$, resulting permutation σ'' = $s_1$,$s_2$….$s_k$ represents a feasible order.
  - Continuing this way σ' can be transformed to σ without violating any deadline.
  - Hence theorem is proved.

- Algorithm GreedyJob(d,J,n)

```
{     J:={1}
      for i:=2 to n do
      {       if (all jobs in J U {i} can be completed by their
deadlines) then J:=J U {i};
      }
}
```

```
Algorithm JS(d,j,n)
// d(i) ≥ 1, 1≤ i ≤ n are the deadlines, n>=1, the jobs are ordered
// such that p1 ≥ p2 ≥ ……. ≥ pn
// In the optimal solution ,d[J(i)] ≤ d[J(i+1)], 1 ≤ i < k
{   d[0] :=J[0]:=0
    J[1]:=1
    k:=1
    for i:=2 to n do
    {     r:=k;
          while((d[J[r]]>d[i]) and (d[J[r]]≠r)) do r:=r-1
          if ((d[J[r]]<=d[i]) and (d[i]>r)) then
          {       for q:=k to (r+1) by -1 do J[q+1]:=J[q]
                  J[r+1]:=i; k:=k+1;
          }
    }return k;
} Complexity is O(n2)
```

72

## JOB SEQUENCING

- **Theorem2:**The Greedy method obtains an optimal solution to the job sequencing problem.
- Proof: Let($p_i$, $d_i$) $1 \leq i \leq n$ define any instance of the job sequencing problem.
- Let I be the set of jobs selected by the greedy method.
- Let J be the set of jobs in an optimal solution.
- Let us assume $I \neq J$ .

73

- If J is contained in I then J cannot be optimal, because less number of jobs gives less profit which is not true for optimal solution.
- Also, I is contained in J is ruled out by the nature of the Greedy method. (Greedy method selects jobs (i) according to maximum profit order and (ii) All jobs that can be finished before dead line are included).
- So, there exists jobs a and b such that $a \in I$, $a \notin J$, $b \in J$,$b \notin I$.
- Let a be a highest profit job such that $a \in I$, $a \notin J$.
- It follows from the greedy method that $p_a \geq p_b$ for all jobs $b \in J, b \notin I$. (If $p_b > p_a$ then the Greedy method would consider job b before job a and include it in I).
- Let $S_i$ and $S_j$ be feasible schedules for job sets I and J respectively.
- Let i be a job such that $i \in I$ and $i \in J$.
  (i.e. i is a job that belongs to the schedules generated by the Greedy method and optimal solution).
- Let i be scheduled from t to t+1 in $S_I$ and $t^1$to $t^1$+1 in $S_j$.
- If $t < t^1$, we may interchange the job scheduled in $[t^1\ t^1+1]$ in $S_I$ with i; if no job is scheduled in $[t^1\ t^1+1]$ in $S_I$ then i is moved to that interval.

74

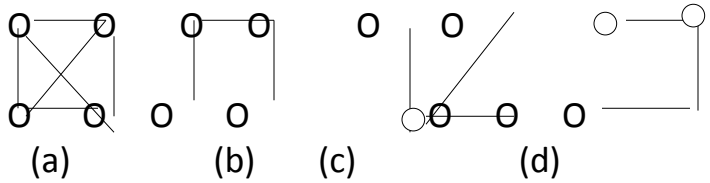
- With this, i will be scheduled at the same time in $S_I$ and $S_J$.
- The resulting schedule is also feasible.
- If $t^1 < t$, then a similar transformation may be made in $S_j$.
- In this way, we can obtain schedules $S_I^1$ and $S_J^1$ with the property that all the jobs common to I and J are scheduled at the same time.
- Consider the interval $[t_a\ t_a+1]$ in $S_I^1$ in which the job a is scheduled.
- Let b be the job scheduled in $S_j^1$ in this interval.
- As a is the highest profit job, $p_a \geq p_b$.
- Scheduling job **a** from $t_a$ to $t_a+1$ in $S_j^1$ and discarding job b gives us a feasible schedule for job set $J^1 = J-\{b\} \cup \{a\}$. Clearly $J^1$ has a profit value no less than that of J and differs I in one less job than does J.
- i.e., $J^1$ and I differ by m-1 jobs if J and I differ from m jobs.
- By repeatedly using the transformation, J can be transformed into I with no decrease in profit value.
- Hence I must also be optimal.

75

# MINIMUM SPANNING TREES

- Definition : Spanning tree : Let G = (V,E) be an un-directed connected graph.

  A minimal connected sub-graph of G which includes all the vertices of G is a spanning tree of G;



## APPLICATIONS OF SPANNING TREES

- Spanning trees can be used to obtain an independent set of circuit equations for an electrical network.Let B be the set of network edges not in the spanning tree. Adding an edge from B to the spanning tree creates a cycle. Different edges from B result in different cycles.
- Kirchoff's second law is used on each cycle to obtain a circuit equation. The cycles obtained in this way are independent as each contains one edge from B which is not contained in any other cycle. Hence the circuit equations so obtained are also independent.

## MINIMUM SPANNING TREES

- Thus a spanning tree is a minimal sub-graph $G^1$ of G such that $V(G) = V(G^1)$ and $G^1$ is connected.
- A connected graph with n vertices must have at least n-1 edges and all connected graphs with n-1 edges are trees.
- If the nodes of G represent cities and the edges represent possible communication links connecting two cities, the minimum number of links needed to connect the n cities is n-1.

## MINIMUM SPANNING TREES (Contd..)

- The spanning trees of G will represent all feasible choices.
- In practice, the edges will have weights associated with them.
- These might represent the cost of construction, the length of the link etc.
- We are interested in finding a spanning tree of G with minimum cost.

Greedy method to obtain a minimum cost spanning tree

Prim's algorithm

- If A is the set of edges selected so far, then A forms a tree.
- The next edges (u,v) to be included in A is a minimum cost edge not in A with the property that A U {u, v} is also a tree.

## PRIM's algorithm

```
Algorithm PRIM (E,cost, n, T)
// E is the set of edges in G cost (n,n) is the adjacency matrix // such as cost (i,j) is a +ve real number or cost (i,j) is ∞ if no // edge (i,j) exists.
// A minimum cost spanning tree is computed and stored as a // set of edges in the array T(1:n-1, 1:2) (T(i,1), T(i,2) is an    // edge in minimum cost spanning tree
{       let (k,l) be an edge of minimum cost in E;//O|E|, O(log|E|)
        mincost := cost[k,l];
        t[1,1]:=k; t[1,2]:=l;
        for i:=1 to n do  //O(n)
                if (cost[i,l]<cost[i,k]) then near[i]:=l;
                else near[i]:=k;
        near[k]:=near[l]:=0;
```

```
        for i:=2 to n-1 do //O(n)
        {       let j be an index such that near[j]≠0 and cost[j,near[j]] is minimum; //O(n), O(log n)
                t[i,1]:=j; t[i,2]:=near[j];
                mincost:=mincost+cost[j,near[j]];
                near[j]:=0;
                for k:=1 to n do // O(n), O|E| overall frequency
                        if ((near[k] ≠0) and(cost[k,near[k]]>cost[k,j]))
                        then near[k]:=j; //O(log n)
        }
        return mincost;
```

# Kruskal algorithm

- High level Kruskal algorithm

1. t=Ø
2. While((t has less than n-1 edges) and (E ≠ Ø))do
3. {
4. Choose an edge (v,w) from E of lowest cost;
5. Delete (v,w) from E;
6. if (v,w) does not create a cycle in t then add (v,w) to t;
7. else discard (v,w);
8. }

# Kruskal Algorithm

- Algorithm Kruskal(E,cost,n,t)

```
//A graph G has n vertices and E edges. //cost[u,v] is the cost of
  edges (u,v). t is the //set of edges in the MST.
{
   Construct a heap out of the edge costs; //O(|E|log|E|)
   for i:=1 to n do parent[i]:=-1; //O(n)
   i:=0; mincost:=0.0;
   while((i<n-1) and (heap not empty)) do //O(n)
   {
       Delete a minimum cost edge (u,v) from the heap and
       reheapify the elements; //O(log |E|)
       j:=Find(u); k:=Find(v);
```

```
        if(j≠k) then
        {       i:=i+1; t[i,1]:=u; t[i,2]:=v;
                mincost:=mincost+cost[u,v];
                Union(j,k);
        }
    }
    if(i ≠ n-1) then write ("No spanning tree");
    else return mincost;
}
Algorithm Union(i,j)
{   p[i]:=j;
}
Alrorithm Find(i)
{   while(p[i]>=0) do i:=p[i];
    return i;
}
```

## **Theorem:** Kruskal's algorithm generates a minimum cost spanning tree

**Proof:** Let G be any undirected connected graph.

- Let T be the spanning tree for G generated by Kruskal's algorithm.
- Let $T^1$ be a minimum cost spanning tree for G.
- We will show that T and $T^1$ have the same cost.

  Let E(T) denote the set of edges in T

  Let E($T^1$) denote the set of edges in $T^1$
- If n is the number of vertices in G, T and T' have n-1 edges.
- If E(T) = E($T^1$) then T is of minimum cost and theorem is proved.

- Assume E(T) ≠ E($T^1$).Then let q be a minimum cost edge such that q∈E(T) and q∉E($T^1$)
- Since $T^1$ is a minimum cost spanning tree, the inclusion of q in $T^1$ creates a unique cycle.
- Let q, $e_1$, $e_2$, $e_3$…. $e_k$ be this cycle.
- At least one of the $e_i$ 1≤ i ≤ k does not belong to E(T); otherwise T will also contain this cycle.
- Let $e_j$ is such that ej∉E(T). So c($e_j$) ≥ c(q).
- If c($e_j$) < c(q) it would have been included into T.
- Also, all the edges in E(T) of cost less than the cost of q are also in E($T^1$) and do not form a cycle with $e_j$..
- Now consider the graph with edge set E($T^1$)U{q}.
- Removal of any edge on the cycle e, $e_1$,$e_2$,….$e_k$ will leave behind a tree $T^{11}$. In particular, if we remove the edge $e_j$ then the resulting tree $T^{11}$ will have a cost no more than the cost of $T^1$.
- Hence $T^{11}$ is also a minimum cost tree.
- By repeatedly using the transformation described above, $T^1$ can be transformed into the spanning tree T without increase in cost. Hence T is a minimum cost spanning tree.

## Single Source Shortest paths problem (Dijkastra Algorithm)

- Let G(V,E) is a directed graph that may represent cities and edges may represent sections of highway. The weights of edges may be distance between two cities or the average time to drive from one city to the other.

**The shortest path problem is as below :**

- Given a directed graph = (V, E) a weighted function c(e) for edges e of G and a source vertex $V_0$.
- The problem is to determine the shortest path from $v_o$ to all the remaining vertices of G where all weights are supposed to be positive
- The paths are generated in increasing order of path length ie. first a shortest path to the nearest vertex is generated then a shortest path to the second nearest vertex is generated and so on.
- So two things need to be determined:
- (i) the next vertex to which a shortest path must be generated and
- (ii) a shortest path to this vertex.

- Let S denote the set of vertices (including $v_o$) to which the shortest paths have already been generated.
- For w $\notin$ S, let Dist(w) be the length of the shortest path starting from $v_o$ going through only those vertices which are in S and ending at w then:

i) If the next shortest path is to vertex u, then the path begins at $v_o$ ends at u and goes through only those vertices which are in S.
  - To prove this, assume there is a vertex w on this path from $V_o$ to u that is not in S. Thus the $v_o$ to u path also contains a path from $v_o$ to w which is of length less than $v_o$ to u path.
  - By assumption the shortest paths are being generated in increasing order of path length and so the shortest path $v_o$ to w must already have been generated. Hence there can be no intermediate vertex which is not in S.

(ii)The destination of the next path generated must be the vertex which has the minimum distance Dist (w) among all vertices not in S.

iii)Having selected a vertex u as in (ii) and generated the shortest $v_o$ to u path, vertex u becomes a member of S.

- The value of Dist (w) may change.
- If it changes, the length of this path is

Dist (u) + c (u, w).

```
Algorithm ShortestPath(v,cost,dist,n)
//dist[j], 1<=j<=n is set to the length of the shortest path from source //vertex to
vertex j. dist[v] is set to zero. cost[1:n,1:n] gives cost //adjacency matrix of graph G. S
is a bit array such that S[i]=0 if vertex // i is not in S, otherwise S[i]=1
{       for i:=1 to n do
        {   S[i]:=false; dist[i]:=cost[v,i];  }
        S[v]:=true; dist[v]:=0.0;
        for num:=2 to n do
        {       Choose u from among those vertices not
        in S such that dist[u] is minimum;
                S[u]:=true;
                for (each w adjacent to u with S[w]=false) do
                        if(dist[w]>(dist[u]+cost[u,w])) then
                                dist[w]:=dist[u]+cost[u,w];
        }
}
```

- Complexity:
  - Any shortest path algorithm must examine each edge in the graph at least once, so minimum possible time would be $\Omega(|E|)$ for any algorithm.
  - Time taken by this algorithm is $O(n^2)$ for a graph having n vertices if cost adjacency matrix is being used to store cost.
  - If edges are maintained as adjacency list and V-S is maintained as red-black tree or heap then overall run time is $O((n+|E|)\log n)$

# DYNAMIC PROGRAMMING

## DYNAMIC PROGRAMMING

- Some problems like knapsack problem, shortest path can be solved by greedy method. In greedy method one optimal decisions is made one at a time.
- For some problems, it is not possible to make stepwise decision in such a manner that the sequence of decisions made is optimal.
- For eg. To find shortest path from vertex i to vertex j, let $A_i$ be vertices adjacent from vertex i. Which of the vertices of $A_i$ should be the second vertex on the path? One way to the solution is to enumerate all decision sequences and pick out the best.
- Enumerating all the solutions will lead to high space and time requirements. Some decision sequences can be avoided that can not be optimal using the principle of optimality.

Principle of optimality:

- An optimal sequence of decisions has the property that whatever the initial state and decisions are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.
- In the greedy method only one decision sequence is generated. In dynamic programming many decision sequences may be generated. Sequences containing suboptimal subsequences can not be optimal so will not be generated.
- For e.g. in shortest path problem, assume that $i,i_1,i_2,i_3\ldots\ldots i_k,j$ is the shortest path from i to j. At the initial vertex i a decision has been made to go to vertex $i_1$. After this decision, problem state is defined by vertex $i_1$ to find path from $i_1$ to j. Sequence $i_1,i_2,i_3\ldots..i_k,j$ must constitute shortest path from $i_1$ to j.
- Another eg. is knapsack problem. In this problem, xi's is restricted to either 0 or 1.

95

- Using KNAP(l,j,y) to represent the problem

$$\text{Maximize } \sum_{l\le i\le j} p_i x_i$$

$$\text{subject to } \sum_{l\le i\le j} w_i x_i \le y, \quad \ldots\ldots\ldots\ldots\ldots\ldots(1)$$

$$x_i = 0 \text{ or } 1 \quad l\le i\le j$$

The 0/1 knapsack problem is KNAP(l,n,M).

Let $y_1,y_2,y_3,y_4, \ldots\ldots y_n$ be the optimal solution.

- If $y_1=0$ then $y_2,y_3\ldots..y_n$ must constitute optimal sequence for KNAP(2,n,m)
- If $y_1=1$, then $y_2,y_3\ldots..y_n$ must constitute optimal sequence for KNAP(2,n,m-$w_1$)
- If it is not, there is another 0/l sequence $z_2,\ldots.,z_n$ such that $\sum p_i z_i$ has greater value. Hence principle of optimality holds.

96

- Let $S_0$ is the initial problem state and n decisions $d_i$ have to be made. Let $D_1=\{r_1,r_2,r_3....r_j\}$ is set of possible decision values for $d_1$. Let $S_i$ be the problem state following the choice of decision $r_i$. Let $\Gamma_i$ be the optimal sequence of decisions with respect to problem state $S_i$. Then optimal sequence of decisions with respect to $S_0$ is the best of the decision sequences $r_i$, $\Gamma_i$, $1<=i<=j$.
- In 0/1 knapsack, let $g_j(y)$ be the value of an optimal solution to KNAP (j+1, n, y).
- Then $g_0(m)$ is the value of an optimal solution to KNAP(l,n,m)
- From the principal of optimality
  $g_0(m) = \max\{g_1(m), g_1(m-w_1) + p_1)\}$
- $g_0(m)$ is the maximum profit which is the value of the optimal solution.

97

- Optimality principle can be equally applied to intermediate states and decisions. Let $y_1,y_2,.....y_n$ be an optimal solution to KNAP(l,n,m).
- Then for each j $l\le j\le n$ , $y_1,..,y_j$ and $y_{j+1},....,y_n$ must be optimal solutions to the problems KNAP$(1,j,\sum_{l\le i\le j} w_i y_i)$
  and KNAP$(j+1,n,m-\sum_{l\le i\le j} w_i y_i)$ respectively
- Then $g_i(y) = \max\{g_{i+1}(y), g_{i+1}(y-w_{i+1}) + p_{i+1}\}....1$
- The above equation is called recurrence equation. Dynamic programming algorithms solve the recurrence to obtain solution.
- We use the knowledge that $g_n(y) = 0$ for all $y>=0$ and $g_n(y) = -\infty$ for $y<0$
- Substituting i = n -1 and using $g_n(y)= 0$ in the above relation(1), we obtain $g_{n-1}(y)$
- Then using $g_{n-1}(y)$, we obtain $g_{n-2}(y)$ and so on till we get $g_0(m)$ which is the solution of the knapsack problem

- Forward and backward approaches to solve recurrence relations:
  - In the forward approach ,decision $x_i$ is made in terms of optimal decision sequences for $x_{i+1}$……..$x_n$ (i.e we look ahead).
  - In the backward approach, decision $x_i$ is made in terms of optimal decision sequences for $x_1$……$x_{i-1}$(i.e we look backward)
- For 0/l knapsack problem
  - $g_j(y)=\max\{g_{i+1}(y), g_{i+1}(y-w_{i+1})+P_{i+1}\}$ is the forward approach as $g_{n-1}(y)$ is obtained using $g_n(y)$
  - $f_j(y) = \max\{f_{j-1}(y), f_{j-1}(y-w_j) + p_j\}$ is the backward approach, as $f_j(y)$ is obtained using $f_{j-1}(y)$
  - $f_j(y)$ is the value of optimal solution to Knap(i,j,y). It can be solved by taking $f_0(y) = 0$ for all $y \geq 0$ and $f_0(y) = -\infty$ for $y < 0$.

99

- Eg. of 0/1 knapsack problem:
  - Let n=3, $w_1$=2,$w_2$=3,$w_3$=4,$p_1$=1,$p_2$=2,$p_3$=5,m=6.
  - We have to compute $g_0(6)$

100

# 0/1 KNAPSACK PROBLEM

- The solution of 0/1 knapsack problem is a sequence of decisions on $x_n, x_{n-1}, \ldots\ldots x_1$ and the total profit; $x_n, x_{n-1}, \ldots. x_1$ may be 0 or 1.
- Let $f_j(y)$ be the value of an optimal solution to KNAP (1, j, y).
- Since the principle of optimality holds

  $f_n(m) = \max \{f_{n-1}(m), f_{n-1}(m-w_n) + p_n\}$
- In general $f_i(y)=\max\{f_{i-1}(y), f_{i-1}(y-w_i)+p_i\}\ldots.(1)$
- This equation can be solved for fn(m) by beginning with $f_0(y)=0$ for all y and $f_i(y)= -\infty$ for $y<0$.

101

- Alternative way is to define sets $S^i$, $S^i_1$ and $S^{i+1}$ as follows.
- $S^i$ is the set of all pairs (p, w) for $f_i$ including (0, 0).
- $S^i_1 = \{(P, W)/ (P-p_i, W-w_i) \in S^i\}$
- $S^{i+1}$ is obtained by merging together $S^i$ and $S^i_1$. (This corresponds to maximum of $f_{i-1}(y)$ and $f_{i-1}(y-w_i)+p_i$.
- Here $S^i$ represents possible states resulting from $2^i$ decision sequences for $x_1, x_2 \ldots x_i$.
- Dominance rule is used to discard dominated tuples. For two pairs $(P_j, W_j)$ and $(P_k, W_k)$ if $P_j <= P_k$ and $W_j >= W_k$ then pair $(P_j, W_j)$ can be discarded.
- All the pairs(P,W) having W>m can also be purged.
- Computing Sn, we can find the solutions to all the problems KNAP(1,n,x), 0<=x<=m.
- Example: n=3, (w1,w2,w3)=(2,3,4), (p1,p2,p2)=(1,2,5) and m=6.

102

- Values of x can be found out by observing the last pair (P1,W1)in $S^n$. If (P1,W1) belongs to $S^{n-1}$ then $x_n$=0 else if (P1-$p_n$,W1-$w_n$) belongs to $S^{n-1}$ then $x_n$=1. This procedure can be repeated n times.

103

# Informal Knapsack Algorithm

```
Procedure DKP (p,w, n,m)
{  S^0= {(0, 0)}
   for i:=1 to n-1 do
   {     S_1^{i-1}:= {(P,W) | P- p_i,W-w_i ∈S^{i-1} and W≤m};
         S^i:= MergePurge (S^{i-1}, S_1^{i-1})
   }
   (PX,WX):= last pair in S^{n-1};
   (PY, WY):= (P'+p_n,W'+w_n) where W' is the largest W in any
   pair in S^{n-1} such that W+w_n<=m;
   if(PX>PY) then x_n:=0;
   else x_n:=1;
   TraceBackFor(x_{n-1},……x_1);
}
```

104

# Formal Knapsack Algorithm

- An array pair[] is being used to represent pairs (P,W). P values are stored in pair[].p and W values are stored in pair[].w. Sets $S^0$,$S^1$… are stored adjacent to each other. b[i] is an array of pointers that gives location of first element in $S^i$.

| | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| pair[].p | 0 | 0 | 1 | 0 | 1 | 2 |
| pair[].w | 0 | 0 | 2 | 0 | 2 | 3 |
| | b[0] | b[1] | | b[2] | | |

- Merging and purging of $S^{i-1}$ and $S_1^{i-1}$ can be carried out at the same time.

105

- Algorithm Dknap(p,w,x,n,m)

```
{   b[0]:=1; pair[1].p:=pair[1].w:=0.0;
    t:=1; h:=1;
    b[1]:=next:=2;
    for i:=1 to n-1 do
    {       k:=t;
            u:= largest q such that t≤q≤h and pair[q].w+w[i]<=m.
            for j:=t to u do
            {       pp:=pair[j].p+p[i]; ww:=pair[j].w+w[i];
                    while((k<=h) and (pair[k].w<ww)) do
                    {       pair[next].p:=pair[k].p;
                            pair[next].w:=pair[k].w;
                            next:=next+1; k:=k+1;
                    }
```

106

```
                    if((k<=h) and (pair[k].w=ww)) then
                    {   if pp<pair[k].p then pp:=pair[k].p;
                        k:=k+1;
                    }
                    if pp>pair[next-1].p then
                    {   pair[next].p:=pp; pair[next].w:=ww;
                        next:=next+1;
                     }
                    while ((k<=h) and (pair[k].p<=pair[next-1].p))
                        do k:=k+1;
            }
            while(k<=h) do
            {       pair[next].p:=pair[k].p;
                    pair[next].w:=pair[k].w; next:=next+1; k:=k+1;
            }
```

107

```
                t:=h+1; h:=next-1; b[i+1]:=next;
        }
        TraceBack(p,w,pair,x,m,n);
}
```

- Complexity of the algorithm:
  - Let $|S^i|$ be the number of pairs in $S^i$, pair should have minimum dimension of $d=\Sigma|S^i|$ where i varies from 0 to n-1.
  - It is difficult to predict next space needed.
  - Each $S^i$ (i>0) is obtained by merging $S^{i-1}$ and $S_1^{i-1}$ and $|S_1^{i-1}| <= |S^{i-1}|$, so $|S^i|<=2|S^{i-1}|$.
  - In the worst case, no pairs will get purged and
    $$\Sigma |S^i| = \Sigma 2^i = 2^n - 1$$
  - Time needed to generate $S^i$ from $S^{i-1}$ is $\Theta(|S^{i-1}|)$, so time needed to compute all the $S^i$'s is $\Theta(\Sigma|S^{i-1}|)$
  - As $|S^i|<=2^i$, time needed to compute all the $S^i$'s is $O(2^n)$.

108

# THE TRAVELING SALESPERSON PROBLEM

- 0/1 Knapsack problem is a subset selection problem. Travelling Salesperson Problem (TSP) is a permutation problem. For n objects subset selection problem requires $2^n$ different subsets of objects whereas for n objects there are n! different permutations and $n! > 2^n$.

TSP problem is:

- Let G = (V,E) be directed graph with edge costs $c_{ij}$. $c_{ij} > 0$ for all i and j and $c_{ij} = +\infty$ if $\langle i,j \rangle \notin E$.
- Let n be the number of vertices ie. $|V| = n$ and $n > 1$.
- A tour of G is a directed cycle that includes every vertex in V. The cost of a tour is the sum of the cost of the edges on the tour. The **TSP** problem is to find a tour of minimum cost.

109

**Applications of TSP**

1. A postal van is to pick up mail from mail boxes located at n different sites. The route taken by a postal van is a tour and the problem is to find a tour of minimum length.
2. A minimum cost tour of a robot arm used to tighten the nuts on some piece of machinery on an assembly line will minimize the time needed for the arm to complete its task.
3. A production environment that produces n different commodities in each production cycle. A sequence of changes from commodity i to commodity j can be followed to minimize change over costs

110

- Let a tour be a simple path that starts and ends at vertex 1.
- Every tour consists of an edge <1,k> for some k $\in$ V-{1} and a path from vertex k to vertex 1.
- The path from vertex k to vertex 1 goes through each vertex in V-{1,k} exactly once.
- If the tour is optimal, then the path from k to 1 must be a shortest k to 1 path going through all vertices in V-{1,k}.
- Hence the principle of optimality holds.
- Let g(i,S) be the length of a shortest path starting at vertex i, going through all vertices in S and terminating at vertex 1.
- Then g(1, V-{1}) is the length of an optimal salesperson tour.

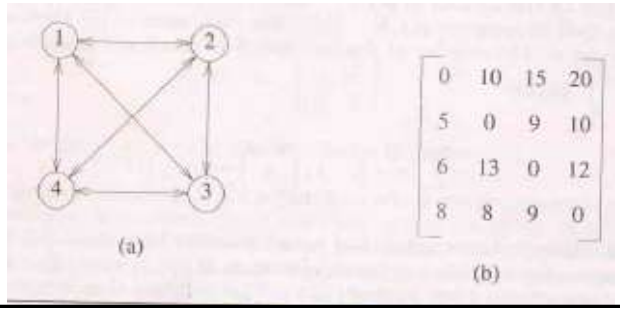
$$g(1, V-\{1\}) = \min_{2 \le k \le n}\{c_{1k} + g(k, V-\{1,k\})\} \quad \ldots\ldots(1)$$

111

- In general, for $i \notin S$,

$$g(i,S) = \min_{j \in s}\{c_{ij} + g(j, S-\{j\})\} \ldots\ldots\ldots(2)$$

- g(1, V-{1}) can be obtained if we know g(k, V-{1,k}) for all choices of k.
- g(k, V-{1,k}) can be obtained from eq. 2
- $g(i,\emptyset) = c_{i1}$ $1 \le i \le n$. $g(i,\emptyset)$ is the length of the shortest path starting at i going through no vertex and ending at 1.
- Then we can use eq.2 to find g(i,S) for all S of size 1, then we can obtain g(i,S) for all S of size 2 and so on…




112

- Algorithm TSP (n,c)

```
{   for i=2 to n do
        g(i,Φ) = c_i1
    for k=1 to n-2
    {   for (all subsets S of (V-{1}) containing k vertices)
        {   for (i such that i ≠ 1 and i is not in S)
            {   g(i,S) = min (c_ij + g(j, S – {j})) where j is some vertex in S
            }
        }
    }
    set min = infinity;
    for k=2 to n by 1 do
    {
        if min > c_1k + g(k,S) where S contains V-{1,k} vertices then
            set min = c_1k + g(k,S)
    }
    return min
}
```

113

- Complexity of TSP
  - Let N be number of g(i,S)'s to be computed before computing g(1,V-{1}).
  - For each value of S there are n-1 choices for i.
  - Number of distinct sets S of size k not including 1 and I is $^{n-2}C_{k.}$
  - So $N= \Sigma(n-1)(^{n-2}C_k) = (n-1)2^{n-2}$

    $0<=k<=n-2$

  This is because according to binomial theorem:

  $(a+x)^n = a^n n_{C0} + a^{n-1} n_{C1} x + a^{n-2} n_{C2} x^2 + \ldots + n_{Cn} x^n$

  If we put a=1, x=1 and n=n-2 then N is obtained as above.
  - Total space required to store g(i,S) is $O(n2^n)$
  - Computation of g(i,S) with $|S|=k$ requires k-1 comparisons so total time complexity is $O(n^2 2^n)$

114

# Optimal Binary Search Tree

A binary search tree is a binary tree; either it is empty or each node contains an identifier and

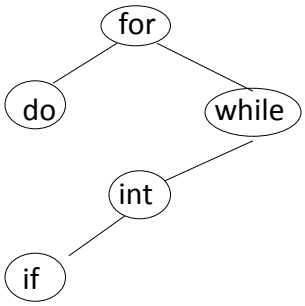
(i) all identifiers in the left sub tree of T are less than the identifiers in the root node T.

(ii) all the identifiers the right sub tree are greater than the identifier in the root node T.

(iii) the right and left sub tree are also BSTs.

115

# Optimal Binary Search trees - Example




In the above BST, if each identifier is searched with equal probability the average number of comparisons for the above tree are $\frac{1+2+2+3+4}{5} = 12/5$.

In general, different identifiers can be searched for with different probabilities.

116

- Unsuccessful searches can also be made.
- Suppose the given set of identifiers is $\{a_1,a_2,\ldots\ldots a_n\}$ with $a_1<a_2<\ldots\ldots<a_n$.
- Let $p_i$ be the probability with which we are searching for $a_i$.
- Let $q_i$ be the probability that identifier x being searched for is such that $a_i<x<a_{i+1}$ $0\le i\le n$, and $a_0=-\infty$ and $a_{n+1}=+\infty$.
- Then $\sum q_i$ is the probability of an unsuccessful search.

$0\le i\le n$

$\sum p(i) + \sum q(i) = 1.$

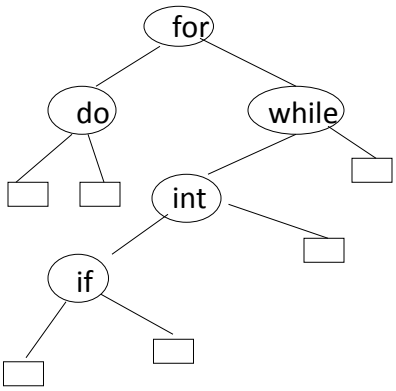
$1\le i\le n \quad 0\le i\le n$

Using this information we need to construct optimal binary search tree $(a_1\ldots\ldots\ldots a_n)$.

- In place of empty sub tree, we add external nodes denoted with squares.
- Internal nodes are denoted as circles.
- A BST with n identifiers will have n internal nodes and n+1 external nodes.

117

# OPTIMAL BINARY SEARCH TREES (Contd..)




118

- Successful search terminates at internal nodes unsuccessful search terminates at external nodes.
- If a successful search terminates at an internal node at level L, then L comparisons are needed.
- Hence the expected cost contribution from the internal nodes for $a_i$ is p(i) * level($a_i$).
- Unsuccessful searches terminates at external nodes.
- The identifiers not in the binary search tree may be partitioned into n+1 equivalent classes

  $E_i$ $0 \le i \le n$.

  $E_o$ contains all X such that $X \le a_1$

  $E_i$ contains all X such that $a_i < X <= a_{i+1}$ $1 \le i \le n$

  $E_n$ contains all X such that $X > a_n$

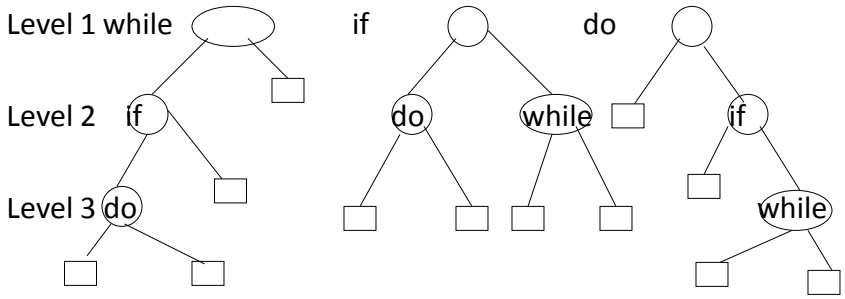
119

- For identifiers in the same class $E_i$, the search terminate at the same external node.
- If the failure node for $E_i$ is at level L, then only L-1 comparisons are made. So the cost contribution of the failure node for $E_i$ is q(i) * (level ($E_i$)-1)
- So the expected cost of a binary search tree is:

  $\sum_{1 \le i \le n} p(i) * level(a_i) + \sum_{0 \le i \le n} q(i) * (level(E_i) - 1)$ …(2)

- An optimal binary search tree for $\{a_1 \ldots\ldots, a_n\}$ is a BST for which eq. 2 is minimum .
- For eg. Let $\{a_1, a_2, a_3\}$={do, if, while}

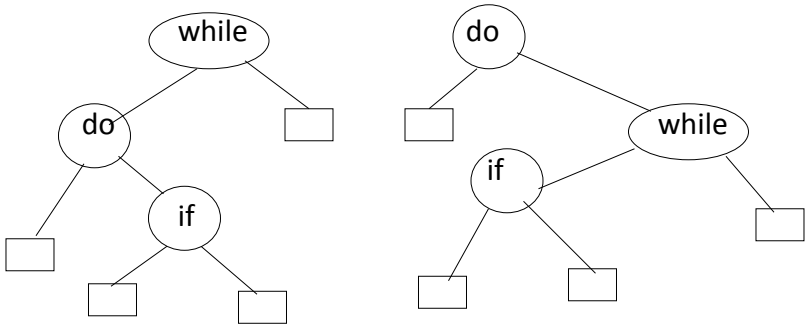
120




(a) (b) (c)

{a1,a2,a3}={do,if,while}

121




(d) (e)

- With equal probability P(i)=Q(i)=1/7, we need to find OBST out of these.
- Cost(tree a)=∑p(i)*level a(i) +∑q(i)*(level (Ei) -1)

  $1 \le i \le n$ $0 \le i \le n$

  (2-1) (3-1) (4-1) (4-1)

  =1/7[1+2+3 + 1 + 2 + 3 + 3 ] = 15/7

tree b is optimal.

122

- If p(1) =0.5, p(2) =0.1, p(3) =0.005 , q(0) =.15 , q(1) =.1, q(2) =.05 and q(3) =.05 find the OBST.
- Cost (tree a) = .5 x 3 +.1 x 2 +.05 x 3 +.15x3 +.1x3 +.05x2 +.05x1 = 2.65
- Cost (tree b) =1.9 , Cost (tree c) =1.5 ,Cost (tree d) =2.05 ,
- Cost (tree e) =1.6 Hence tree C is optimal.

To obtain a OBST using Dynamic programming we need to take a sequence of decisions regarding the construction of tree.

- First decision is which of $a_i$ is to be the root.
- If $a_k$ is chosen as the root, then the internal nodes for $a_1,\ldots\ldots,a_{k-1}$ and the external nodes for classes $E_o,E_1,\ldots\ldots,E_{k-1}$ will lie in the left subtree L of the root.
- The remaining nodes will be in the right subtree R.

123

$$\text{Cost}(l) = \sum_{1\le i<k} p(i)*\text{level}(a_i) + \sum_{0\le i<k} q(i)*(\text{level}(E_i)-1)$$

$$\text{Cost}(r) = \sum_{k<i\le n} p(i)*\text{level}(a_i) + \sum_{k<=i\le n} q(i)*(\text{level}(E_i)-1)$$

- Here $t_{ij}$ is the tree with nodes $a_{i+1},\ldots\ldots,a_j$ and nodes corresponding to $E_i,E_{i+1},\ldots\ldots,E_j$.
- Let w(i,j) represents the weight of tree $T_{ij}$.

  $w(i,j)=p(i+1)+p(i+2)+..+p(j)+q(i)+q(i+1)\ldots q(j)=q(i)+\sum_{l=i+1}[q(l)+p(l)]$
- The expected cost of the search tree is

  $p(k)+\text{cost}(l)+w(0,k-1)+\text{cost}(r)+w(k,n)\ldots\ldots\ldots(1)$

  w(0,k-1) is the sum of probabilities corresponding to nodes and nodes belonging to equivalent classes to the left of $a_k$.

  w(k,n) is the sum of the probabilities corresponding to those on the right of $a_k$.

124

- If Tree is optimal then eq. 1 must be minimum over all binary search trees containing $a_1,\ldots a_{k-1}$ and $E_0,E_1,\ldots..,E_{k-1}$
- Similarly cost(r) must also be minimum.
- Let c(i,j) represent the cost of the tree $t_{ij}$ (containing nodes $a_{i+1},\ldots\ldots a_j$ and $E_i,\ldots..,E_j$).
- Then cost(l) = c(0,k-1) and cost(r)=c(k,n).
- Then expected cost of the search tree is:
  p(k)+c(0,k-1)+c(k,n)+w(0,k-1)+w(k,n).....(2) which is minimum if k is chosen such that eq. 2 is minimum.
- c(0,n)=min{c(0,k-1)+c(k,n)+p(k)+w(0,k-1)+w(k,n)}.....(3)
  $1<=k\leq n$
- Substituting i instead of 0 and j instead of n in (3)
  c(i, j) = min { c(i, k-1)+ c (k, j)+ p(k) + w(i, k-1) + w (k, j)}
  $i<k\leq j$
  = min {c(i,k-1)+c(k,j)}+w(i,j).....(4)
  $i<k\leq j$
- Also w(i,j)=p(j)+q(j)+w(i,j-1)

125

- Eq. 4 can be solved for c(0,n) by first computing c(i,j) such that j-i=1.
- c(i,i)=0 and w(i,i)=q(i),$0\leq i\leq n$
- Then we can compute all c(i,j) such that j-i = 2, then all c(i,j) with j-i=3 and so on.
- During these computations, the root r(i,j) of each tree Tij is recorded, and OBST can be constructed using this information.

- Example:
  - Let n=4
  - (a1,a2,a3,a4)=(do,if,int,while)
  - p(1:4)=(3,3,1,1)
  - q(0:4)=(2,3,1,1,1)

126

```
• Algorithm OBST(p,q,n)
  {     for i:=0 to n-1 do
        {       w[i,i]:=q[i]; r[i.i]:=0; c[i,i]:=0.0;
                w[i,i+1]:=q[i]+q[i+1]+p[i+1];
                r[i,i+1]:=i+1;
                c[i,i+1]:=q[i]+q[i+1]+p[i+1]
        }w[n,n]:=q[n];r[n,n]:=0;c[n,n]:=0.0;
        for m:=2 to n do
        {       for i:=0 to n-m do
                {       j:=i+m;
                        w[i,j]:=w[i,j-1]+p[j]+q[j];
                        k:=Find(c,r,i,j);
                        c[i,j]:=w[i,j]+c[i,k-1]+c[k,j];
                        r[i,j]:=k;
                }
        } write (c[0,n],w[0,n],r[0,n]);
  }
```

127

```
• Algorithm Find(c,r,i,j)
  {
        min:=∞;
        for m:=i+1 to j do
        {       if (c[i,m-1]+c[m,j])<min then
                {
                        min:=c[i,m-1]+c[m,j];
                        l:=m;
                }
        }
        return l;
  }
```

128

- Complexity of OBST
  - We need to compute c(i,j) for (j-i) = 1,2,3,…n.
  - When j-i=m, there are n-m+1 c(i,j)'s to compute.
  - Computation of each c(i,j) require us to find the minimum of m quantities. So each c(i,j) can be computed in O(m) time.
  - Total time for all c(i,j)s with j-i=m is $O(nm-m^2)$
  - Total time to evaluate all c(i,j)s is $O(n^3)$

129

# BACKTRACKING

# BACKTRACKING

- Desired solutions of certain problems are expressible as n-tuple $(x_1,\ldots,x_n)$, where the $x_i$ are chosen from some finite set $S_i$
- Problem is to find a vector that maximizes or minimizes a criterion function $P(x_1 , \ldots . , x_n)$.
- Example:
  - Sorting an array a[1:n].
  - Solution of this problem is a vector $(x_1,\ldots,x_n)$ where $x_i$ is the index in a of the ith smallest element.
  - Criterion function P is the inequality $a[x_i]<=a[x_{i+1}]$ for 1<=i<n.
  - Set $S_i$ includes integers 1 through n.

131

- Suppose $m_i$ is the size of set $S_i$. Then $m=m_1m_2\ldots..m_n$ tuples are possible candidates for satisfying the function P. The brute force approach would be to form all of these n-tuples and evaluate each one with P, finding the optimum. Backtracking algorithm can find the same answer with few number of trials.
- Major advantage in backtracking algorithm is bounding function.
- Two types of constraints can be defined for the problems: explicit and implicit.
- Explicit constraints: Rules that restrict each $x_i$ to take on values only from the given set. All tuples that satisfy explicit constraints define a possible solution space for a problem instance.
- Implicit constraints: Rules that determine which tuples in solution space satisfy the criterion function.

132

Example : 8 queens problem

- The problem is to place eight queens on an 8 x 8 chess board so that no two queens attack i.e. no two of them are on the same row, column or diagonal.
- Strategy : The rows and columns are numbered through 1 to 8.
- The queens are also numbered through 1 to 8.
- Since each queen is to be on a different row we assume queen i is to be placed on row i.
- The solution is an 8 tuple $(x_1,x_2,.....,x_8)$ where $x_i$ is the column on which queen i is placed.
- The explicit constraints are :

  $S_i = \{1,2,3,4,5,6,7,8\}$ $1 \leq i \leq n$ or $1 \leq x_i \leq 8$
  i = 1,.....8
- The solution space consists of $8^8$ 8- tuples.

133

The implicit constraints are :

(i) no two $x_i$s can be the same.

(ii) no two queens can be on the same diagonal.

(i) reduces the size of solution space from $8^8$ to 8!

8 – tuples.

Two solutions are (4,6,8,2,7,1,3,5) and (3,8,4,7,1,6,2,5)

134

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 1 | | | | Q | | | | |
| 2 | | | | | | Q | | |
| 3 | | | | | | | | Q |
| 4 | | Q | | | | | | |
| 5 | | | | | | | Q | |
| 6 | Q | | | | | | | |
| 7 | | | Q | | | | | |
| 8 | | | | | Q | | | |

135

- Backtracking algorithms determine solutions by systematically searching solution space.
- Tuples that satisfy the explicit constraints define a solution space that can be organized into a tree.
- Each node in the tree defines a problem state.
- All paths from the root to other nodes define the state-space of the problem.
- Solution states are the states leading to a tuple in the solution space.
- Answer nodes are those solution states leading to an answer-tuple( i.e. tuples which satisfy implicit constraints).
- Tree organization of the solution space is called state space tree.
- For eg. in n-queens problem which is a generalization of 8 queens problem, the solution space consists of n! permutations. A tree can be generated for n=4, where edges from level i to level i+1 are labeled with the values of $x_i$.

- Terminology used:
  - Live node: A node which has been generated and all of whose children are not yet been generated .
  - E-node: The live node whose children are currently being generated .
  - Dead node: A node that is either not to be expanded further, or for which all of its children have been generated.
  - Depth first generation: In this method, as soon as a new child C of the current E-node R is generated, C will become the new E-node. R will become E-node again when C has been fully explored.
  - Another method is where E-node remains E-node until it is dead.
  - Bounding function: This function is used to kill live nodes without generating all their children.
  - Backtracking is depth first generation with bounding functions.
  - Branch and Bound: It is a method in which E-node remains E-node until it is dead.
  - How backtracking solves 4-queens problem: exercise.

# Backtracking-General Method

- Assume that all the answer nodes are to be found.
- Let $(x_1,x_2,x_3,.....x_i)$ be a path from root to a node.
- Let $T(x_1,x_2,.....x_i)$ be the set of all possible values for $x_{i+1}$. $T(x_1,x_2,.....x_n)=\emptyset$
- $B_{i+1}$ is a bounding function such that if $B_{i+1}(x_1,x_2...x_{i+1})$ is false for a path $(x_1,x_2....x_{i+1})$ from root node to a problem state, then the path cannot be extended to reach an answer node. Hence those values which are generated by T and satisfy $B_{i+1}$ are the candidates for position i+1.

- Algorithm Backtrack(k)

```
// using recursion. x[] and n are global.
{   for each x[k] Є T(x[1],…..x[k-1]) do
    {       if (Bk(x[1],x[2],….x[k]) ≠ 0)) then
            {       if (x[1],x[2],….x[k] is a path to an
                    answer node) then write (x[1:k];
                    if (k<n) then Backtrack(k+1);
            }
    }
}
```

This algorithm is invoked by Backtrack(1);

- Algorithm Backtrack(n)

```
// using iteration.
{   k:=1;
    while(k ≠ 0) do
    {       if (there remains an untried x[k]Є T(x[1],….. x[k-
            1])) then
            {   if(Bk(x[1],x[2],….x[k]) is true) then
                {
                    if (x[1],x[2],….x[k] is a path to an answer
                    node) then write (x[1:k];
                    k:=k+1;

            }
            else k:=k-1;
    }
}
```

- Efficiency of backtracking algorithm depends on four factors:

  (i) The time to generate the next x(k);

  (ii) The number of x(k) satisfying the explicit constraints

  (iii) The time for bounding functions $B_k$

  (iv) The number of x(k) satisfying the $B_k$.

Once a state space tree organization is selected, first three factors are independent of problem instance, fourth factor varies from one problem instance to another.

- A good bounding function reduces number of nodes generated. A trade-off is involved. Good bounding functions need more time for evaluation.

- A backtracking algorithm might generate O(n) nodes for one problem instance and O(n! or $2^n$) nodes for another instance. So the worst case complexity could be $O(p(n)2^n)$ or O(q(n)n!) where p(n) and q(n) are polynomials in n.
- One general principle of efficient searching is *rearrangement*.
- Estimation of nodes generated (using Monte Carlo Method):
  - Generate a random path in the state space tree.
  - Let X be a node at level i on this path.
  - Let $m_i$ be the children of X ( at level i+1 ) that do not get bounded. (i.e. $m_i$ are the nodes which can be considered for getting an answer node ).
  - Choose randomly one of the $m_i$.
  - Continue this until this node is either a leaf or all its children are bounded.

- Let m be the no. of unbounded nodes to be generated.
- Suppose bounding functions are static, i.e., the backtrack algorithm does not change its bounding functions.
- The number of estimated number of unbounded nodes $m=1+m_1+m_1m_2+\ldots. +m_1m_2m_3..m_{i\ldots.}$ .
  - » The number of unbounded nodes on level one is 1.
  - » The number of unbounded nodes on level 2 is $m_1$
  - » If the nodes on the same level have same degree, then each level 2 node will have on average $m_2$ unbounded children. It means there are total $m_1m_2$ nodes at level 3 and so on.
  - » $\therefore$ The total number of nodes generated at level i+1 is $m_1 m_2 \ldots m_i$ .
- The above procedure can be written as an algorithm.

- Algorithm Estimate()

```
{   k:=1;m:=1;r:=1;
    repeat
    {   Tk:={x[k]|x[k]ЄT(x[1],x[2],….x[k-1]) and
        Bk(x[1],…x[k]) is true};
        if(Size(Tk) =0) then return m;
        r:=r*Size(Tk);
        m:=m+r;
        x[k]:=Choose(Tk);
        k:=k+1;
    } until (false);
}
```

# The 8-Queens Problem

- Here we need to find ($x_1$,$x_2$…$x_8$) ie. positions of $Q_1$,$Q_2$…$Q_8$.
- ith queen is to be placed in ith row and values in x's should be distinct.
- To check diagonal condition:
  - Every element on the same diagonal that runs from upper left to lower right has same row-column value.
  - Every element on the same diagonal that goes from upper right to lower left has the same row+column value.
  - Two queens placed at positions (i,j) and (k,l) are on same diagonal if: i-j=k-l or i+j=k+l or |j-l|=|i-k|.

- Algorithm Place(k,i) returns true if kth queen can be placed in column i.
- Algorithm Place(k,i)

```
{   for j:=1 to k-1 do
        if((x[j]=i) or (Abs(x[j]-i)=Abs(j-k))) then
            return false;
    return true;
}
```

Computing time of this algorithm is O(k-1)

- Algorithm NQueens(k,n)

```
{   for i:= 1 to n do
    {       if Place(k,i) then
            {
                    x[k]:=i;
                    if (k=n) then write (x[1:n]);
                    else NQueens(k+1,n);
            }
    }
}
```

This algorithm is invoked by Nqueens(1,n);

Total number of nodes in 8-queens state space tree is

$$1+\Sigma_{j=0 \text{ to } 7}[\Pi_{i=0 \text{ to } j}(8-i)]=69,281$$

Average value Estimate algorithm produces is 1625 which is only 2.34% of total number of nodes.

# Graph Coloring

- m-colorability dicision problem: If the nodes of a given graph G can be colored using m colors in such a way that no two adjacent nodes have the same color.
- If d is the degree of the given graph, it can be colored with d+1 colors.
- m-colorability optimization problem: To find the smallest integer m for which the graph G can be colored. This m is called chromatic number of the graph.
- A graph is said to be planar iff it can be drawn in a plane in such a way that no two edges cross each other.
- A famous problem is 4-color problem for planar graphs: Given any map, can the regions be colored in such a way that no two adjacent regions save the same color yet only four colors are needed.

- Suppose a graph is being represented by its adjacency matrix G[1:n,1:n] where G[i,j]=1 if (i,j) is an edge of G and G[i,j]=0 otherwise.
- Colors are represented by integers 1,2,….m.
- Solution of the problem is to find n-tuple($x_1$,$x_2$,…$x_n$) where $x_i$ is the color of node i.
- A state space tree used is a tree of degree m and height n+1. Example: When n=3 and m=3.

```
Algorithm mColoring(k)
{     repeat
      {       NextValue(k);
              if(x[k]=0) then return;
              if(k=n) then write(x[1:n]);
              else mColoring(k+1);
      }until(false);
}
Algorithm NextValue(k)
{     repeat
      {       x[k]:=(x[k]+1)mod (m+1);
              if(x[k]=0) then return;
              for j:=1 to n do
                      if((G[k,j] ≠ 0) and (x[k]=x[j])) then break;
              if(j=n+1) then return;
      }until(false);
}
```

- Complexity of mColoring Algorithm is $O(nm^n)$
  - Total number of internal nodes in state space tree is $\Sigma_{(i=0 \text{ to } n-1)} m^i$
  - At each internal node O(mn) time is required.
  - Total time: $\Sigma_{(i=0 \text{ to } n-1)} m^{i+1}n = \Sigma_{(i=1 \text{ to } n)} m^i n = n(m^{n+1}-2)/(m-1)$
  - $=O(nm^n)$

# Hamiltonian Cycles

- Let G=(V,E) be a connected graph with n vertices . A Hamiltonian cycle is a round path along n edges of G which visits every vertex once and returns to its starting position. We need to find $v_1, v_2, \ldots v_{n+1}$ where all $v_j$ are distinct except $v_1$ and $v_{n+1}$ and there is an edge from $v_i$ to $v_{i+1}$
- The backtracking solution is a vector $(x_1, \ldots\ldots, x_n)$ where $x_i$ represents the $i^{th}$ visited vertex of the cycle.
- We initialize Graph (1:n,1:n) and X(2:n)=0, X(1)=1 and start with HAMILTONIAN (2).

```
Algorithm Hamiltonian(k)
{       repeat
        {       NextValue(k);
                if(x[k]=0) then return;
                if(k=n) then write(x[1:n]);
                else Hamltonian(k+1);
        }until(false);
}
Algorithm NextValue(k)
{       repeat
        {       x[k]:=(x[k]+1)mod (n+1);
                if(x[k]=0) then return;
                if(G[x[k-1],x[k]] ≠ 0) then
                {       for j:=1 to k-1 do if(x[j]=x[k]) then break;
                        if(j=k) then
                           if((k<n) or ((k=n) and G[x[n],x[1]] ≠ 0)) then return;
                }
        }until(false);
}
```

# BRANCH AND BOUND

# BRANCH AND BOUND

- Branch and bound is a state space search method in which all the children of E node(current node) are generated before any other live node becomes the E-node.
- Two types of branch and bound strategies:
  - FIFO search (based on BFS)
  - LIFO search (based on D search)
- Example: Solving 4 queens problem using FIFO search.

155

# Least Cost (LC)- SEARCH

- In FIFO and LIFO Branch & Bound, the selection rule for the next E-node is rigid. It does not give any preference to a node that has a very good chance of getting the search to an answer node quickly.
- The expansion of all live-nodes is required before the node leading to an answer node.
- The search for an answer node can be speeded up by using an intelligent ranking function ĉ(.) for live nodes.
- In 4 queens problem, if a ranking function is used that assigns a node leading to answer node a better rank than that node can be expanded early before expanding remaining nodes.

156

- ĉ(x) is the estimated additional computational effort or the additional cost needed to reach an answer node from the live node x.
- The cost of a node x could be (i) the number of nodes in the subtree that need to be generated before an answer node is generated or (ii) the number of levels the nearest answer node is from x.
- If cost measure (i) is used then the search would always generate the minimum number of nodes every branch and bound algorithm must generate.
- Example: 4 queens problem.
- As c(x) is not known in advance, rank can be assigned on the basis of estimate only.

157

- C (x) is defined as: ĉ(x)=f(h(x))+ĝ(x)
- Here ĝ(x) is an estimate of additional effort needed to reach an answer node from x.
- h(x) is the cost of reaching x from the root and f(.) is any nondecreasing function.
- Using f(.)=0, biases the search algorithm. Normally ĝ(y)<= ĝ(x) where y is a child of x. If we take f(.)=0, then only one branch may be explored leaving other branches that may lead to better answer.
- A search strategy that uses a cost function ĉ(x) = f(h(x)) + ĝ (x) to select the next E-node, would always choose for its next E-node, a live node with least ĉ(.). Hence such a strategy is called Least cost search or LC-Search

158

- BFS and D search are special cases of LC-Search
- LC search with $\hat{g}(x)=0$ and $f(h(x))$ = level of x is BFS. LC search with $f(h(x))=0$ and $\hat{g}(x) >= \hat{g}(y)$ whenever y is a child of x is D search.
- LC-Search with bounding functions is LC branch and bound search.
- The Actual cost of X is denoted with c(X).
- Thus $\hat{c}(X)$ is an estimation of C(X).
- An LC-search expands the node with minimum cost $\hat{c}(X)$ .
- But it is not necessary that LC-search always finds an answer node with minimum cost.
- For two nodes X and Y with $\hat{c}(X) < \hat{c}(Y)$, it may be possible that $c(X)>c(Y)$.
- Even if $c(X)<c(Y)$ it may also be possible that a minimum cost answer node may not be found.

159

- Example: 15 puzzle problem:
  - This puzzle consists of 15 numbered tiles on a square frame with capacity of 16 tiles. Initial arrangement of tiles is to be transformed into the goal arrangement using legal moves only. Legal move is the one in which a tile adjacent to the empty spot is moved to empty spot.
  - There are 16! different arrangements of the tiles.
  - A state in the state space tree is reachable if there is a sequence of legal moves from the initial state to this state. It could be tested whether the goal state is reachable from the initial state or not.
    - Let less(i) be the number of tiles j such that $j<i$ and $position(j)>position(i)$. Let $x=1$ if in the initial state the empty spot is at one of the shaded positions.
    - Then the goal state is reachable from the initial state iff $\Sigma_{(i=1 \text{ to } 16)}$ less(i) +x is even.

160

- State space tree can be designed using the following rules:
  - Empty space is moved either up, right, down or left.
  - No node p has a child state that is the same as p's parent.
  - FIFO, LIFO or LC search can be applied.
  - In LC search, an intelligent search method can be included.
    - A cost c(x) can be associated with each node x in the state space tree. It can be the length of a path from the root to a nearest goal node. So only those nodes along the path to the goal state will be generated. However it is not possible to compute function c(.). But we can compute $\hat{c}(x)$, an estimate of c(x) where $\hat{c}(x)=f(x)+\hat{g}(x)$. f(x) is length of the path from the root to node x and $\hat{g}(x)$ is estimate of the length of a shortest path from x to a goal node. One possible choice of $\hat{g}(x)$ could be: number of nonblank tiles not in their goal position (it means atleast $\hat{g}(x)$ moves need to be made)
    - $\hat{c}(x)$ is a lower bound on the value of c(x)

161

# Control abstraction for LC search

```
• Algorithm LCSearch(t)
  {     if *t is an answer node then output *t and return;
        E:=t;
        Initialize the list of live nodes to be empty;
        repeat
        {   for each child x of E do
            {     if x is an answer node then output the path x to t and return;
                  Add(x);
                  (x->parent):=E;
            }
            if there are no more live nodes then
            {     write ("No answer node") and return;}
            E:=Least();
        }until(false);
  }
```

162

- LC Search terminates only when either an answer node is found or the entire state space tree has been generated and searched ie. LC Search terminates for finite state space trees. For infinite state space trees, LC search may not terminate. It is a good idea to use upper bound on the cost of the node.
- Similarity between LC search, BFS and D Search:
  - If list of live nodes is implemented as a queue then LC search will behave as FIFO search.
  - If list of live nodes is implemented as a stack then LC search will behave as LIFO search.

163

# Bounding

- $\hat{c}(x)$ is used to provide lower bounds on solutions obtainable from node x. If upper(U) is an upper bound on the cost of a minimum cost solution then all live nodes x with $\hat{c}(x)>upper(U)$ may be killed.
- Starting value of upper can be obtained by some heuristic or can be set to $\infty$.
- Each time a new answer node is found, value of U may be updated.
- Minimization problems can be solved using LC search. To solve maximization problem, sign of objective function can be changed to change the problem to minimization problem.

164

- For calculating c(.), objective function itself can be used. For feasible solutions, c(x) is the value of objective function and for infeasible solutions, c(x)=∞.
- For partial solution, c(x) is the cost of minimum cost node in the subtree with root x. As c(x) is difficult to compute, ĉ(x) is used.
- Example: Job sequencing with deadlines:
  - We are given n jobs and one processor.
  - Job i has associated with it three tuple (pi,di,ti). ti is the units of processing time. If processing is not completed by the deadline di, then penalty pi is incurred.
  - Objective is to select a subset J of n jobs such that all jobs in J can be completed by their deadlines. So penalty will be incurred only on those nodes which are not in J. This penalty should be minimum.
  - This problem can be solved using FIFOBB and LCBB

165

- Problem: n=4, $(p_1,d_1,t_1) = (5,1,1)$, $(p_2,d_2,t_2) = (10,3,2)$, $(p_3,d_3,t_3) = (6,2,1)$, $(p_4,d_4,t_4) = (3,1,1)$

166

- Knapsack problem is a maximization problem. It is to be converted to minimization problem:
  - Minimize $-\Sigma p_i x_i$
  - Subject to $\Sigma w_i x_i <= m$ and $x_i = 0$ or 1 where $1 <= i <= n$
  - Two organization of state space trees:
    - Variable size tuple
    - Fixed size tuple
  - Fixed size tuple tree is being considered where every leaf node for which $w_i x_i <= m$ is an answer node. All other nodes are infeasible. $c(x) = -\Sigma_{(i=1 \text{ to } n)} p_i x_i$ gives cost of every answer node. For infeasible nodes $c(x) = \infty$. c(x) is recursively defined as min{c(lchild(x)), c(rchild(x))}
  - Lower bound (ĉ)and upper bound(u) can be calculated using the algorithms bound and ubound.

167

# 0/1 Knapsack

- Algorithm Ubound(cp,cw,k,m)

```
{       b:=cp; c:=cw;
        for i:=k+1 to n do
        {           if(c+w[i]<=m) then
                            c:=c+w[i]; b:=b-p[i];
        }
        return b;
}
```

- Algorithm Bound(cp,cw,k)

```
{       b:=cp; c:=cw;
        for i:=k+1 to n do
        {           c:=c+w[i];
                    if(c<m) then b:=b-p[i];
                    else return b-(1-(c-m)/w[i])*p[i];
        }return b;
}
```

168

- LC BB solution for given problem:
  - n=4.
  - (p1,p2,p3,p4)=(10,10,12,18).
  - (w1,w2,w3,w4)=(2,4,6,9)
  - m=15.
- To solve the problem, we need to specify
  - The structure of nodes in the state space tree
  - How to generate children of given node
  - How to recognize a solution node
  - Mechanism for adding a node to list of live nodes and identifying least cost node.
- Extra variables that can be associated with the nodes are:
  - Parent, Tag, Level, Cu (Capacity unused), Pe (Profit earned), Ub (Upper bound)
- In the same way FIFOBB can be used to solve knapsack problem.

169

# Travelling Salesperson problem

- Let G = (V,E) be a directed graph representing cities and direct edges between cities.
- Let $C_{ij}$ = cost of edge (i,j), $C_{ij} = \infty$ if $(i,j) \notin E$ and n is the total number of nodes.
- Let every tour starts & ends at vertex 1.
- So, the solution space

  $S = \{ 1, \pi, 1 \mid \pi \text{ is a permutation of } (2,3,.....n)\}; |S| = (n-1)!$
- The size of S may be reduced by restricting S so that $(1,i_1,i_2,.. i_{n-1}, 1) \in S$ iff $<i_j, i_{j+1}> \in E$, $0 \leq j \leq n-1$, $i_0 = i_n = 1$.
- S can be organised into a state space tree.
- For eg. complete graph with $|V| = 4$.
- Each leaf node (L) is a solution node & defines a path from root to L.

170

- To use LCBB to search travelling salesperson state space tree, we need to define a cost function c(.) and two other functions ĉ(.) and u(.) such that

  $\hat{c}(x) \leq c(x) \leq u(x)$ .
- The cost c(.) is such that the solution node with least c(.) corresponds to a shortest path/tour in G one choice for c(.) is :
- C(A) = { length of tour defined by path from root to A, if A is a leaf.

  Cost of minimum-cost leaf in the sub tree A, if A is not a leaf.

  ĉ(.) can be obtained by using the reduced cost matrix corresponding to G.
- A row (column) is said to be reduced iff it contains at-least one zero and all remaining entries as non-negative.
- A Matrix is reduced iff every row and column is reduced.
- The cost reduced will be the lower bound of the root.
- A reduced cost matrix is associated with each node in the tree.

171

- If A is the reduced cost matrix for node R and S be a child of R such that tree edge <R,S> is included then reduced cost matrix for S can be obtained as:
  - Change all entries in row i and column j of A to ∞.
  - Set A(j,1) to ∞.
  - Reduce all the columns and rows.
- Cost of S can be obtained as ĉ(S)=ĉ(R)+A(i,j)+r
- Example.

172

# Dynamic State Space organization for the problem

- If G = (V,E) has e edges then every tour contains exactly n edges.
- For each vertex i, there is exactly one edge of the form (i,j) and one of the form (k,i) in every tour.
- A possible organisation for the state space tree is a binary tree in which left branch represents the inclusion of a particular edge and the right branch represents the exclusion of that edge.
- If edge <i,j> is used then the left sub-tree of the root will represent all tours including edge <i,j> & the right sub tree will represent all tours that do not include edge <i,j>.
- If an optimal tour is included in the left sub tree then only (n-1) edges need to be selected.
- If all optimal tours lie in the right sub tree then we have still to select n edges.

173

- Since the left sub tree selects fewer edges, it will be easier to find an optimal solution in it than to find one in the right sub tree.
- Selection Rule for edges is to select that edge which results in a right sub tree that has highest ĉ value.
- Considering the example, at the root node, if we select an edge <i,j> whose cost in reduced matrix is positive, then the ĉ value of the right sub tree will remain 25.
- This is because the reduced matrix for right sub-tree will have cost (i, j) = ∞ & all other entries are identical.
- Hence ĉ cannot increase.
- So, we must choose an edge with reduced cost 0. Once this is chosen, the cost of the edge will be set to ∞ and the matrix may be reduced.
- To reduce the matrix again some amount is to be subtracted and it is to be added to the ĉ value. Thus the ĉ value may increase

174

- If A is the reduced cost matrix for node R, then the selection of edge <i,j> (A(i,j) = 0) as the next partitioning edge will increase the ĉ of the right sub-tree by

  $\Delta$ = min k≠j { A (i,k) } + min k≠i{ A(k,j) }

  as this much needs to be subtracted from row i & column j to introduce a zero into both.
- Each time we selects an edge having maximum value of $\Delta$.
- Selecting an edge <i,j> dynamically requires setting ith row equal to 0, jth column equal to 0 and one more entry equal to zero that may lead to a cycle.
- Example

175

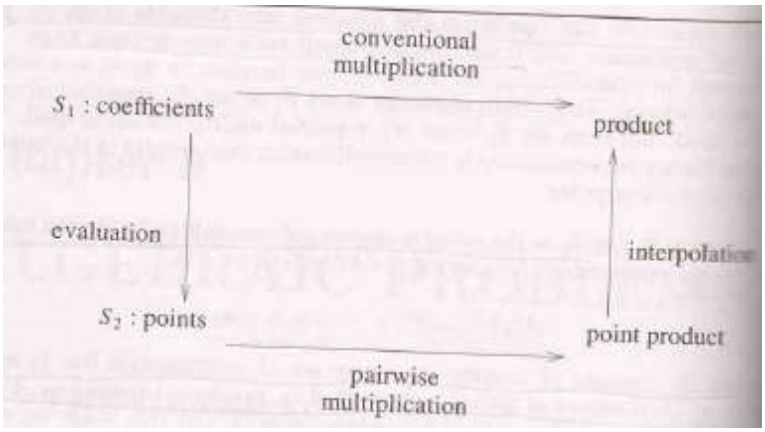
# LOWER BOUND THEORY

# INTRODUCTION TO ALGEBRAIC PROBLEMS

- Computers are not directly equipped to manipulate symbolic mathematical expressions such as polynomials.
- A system that allows manipulation of mathematical expressions is called mathematical symbol manipulation system.
- First design technique is algebraic transformation. Suppose input I is a member of set $S_1$ and f(I) is to be computed. Method for computing f(I) may be inefficient. So transform the input into another form to produce member of set $S_2$. $S_2$ contains exactly same elements as $S_1$ but with different representation. It may be easier to compute function for a member of $S_2$. After computing the result reverse transformation can be performed.
- Eg.1 Operation on decimal number by converting it into binary.

177

- Eg.2 $S_1$ set of n degree polynomials with integer coefficients:
  - $A(x)=a_nx^n+.....a_1x+a_0$
  - Set $S_2$ consists of exactly same set of polynomials but represented by their values at 2n+1 points ie. 2n+1 pairs $(x_i,A(x_i))$
  - Suppose function is to determine product of two polynomials.
  - Complexity of conventional method is $O(n^2)$.




178

- In the equivalent method of evaluation, pairwise multiplication and interpolation, pairwise multiplication will take O(n) time and a polynomial of degree 2n can be obtained using interpolation as 2n+1 points are there.

179

# LOWER BOUND THEORY

- The main purpose in the design of algorithms for problem solving is to obtain a correct and efficient solution.
- If two algorithms for solving the same problem is there then the one with smaller order of complexity is superior. But search for more faster algorithm continues.
- The purpose of the lower bound theory is to demonstrate a few techniques that have been used to establish that a given algorithm is the most efficient possible ie. a function is discovered g(n) that is a lower bound on the time that any algorithm must take to solve the given problem.
- If there is an algorithm whose computing time is the same order as g(n), it is taken as most efficient one.

180

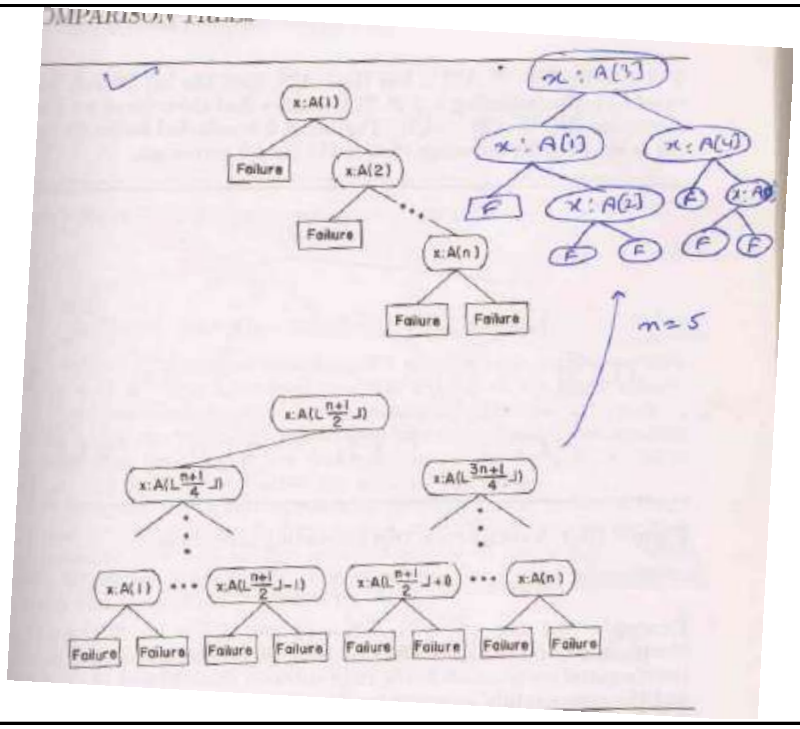
- Big Omega notation is used to express lower bounds.
- f(n)=Ω(g(n)) means that g(n) is the lower bound for f(n) ie. f(n)>=c*g(n) for all $n>n_0$.
- Deriving good lower bounds is often more difficult than devising efficient algorithms as a lower bound states a fact about all possible algorithms for solving a problem.
- For many problems lower bound can easily be observed. For example all the algorithms that finds maximum of an unordered set of n integers has a lower bound of Ω(n).
- Multiplication of two n*n matrices has a lower bound of $\Omega(n^2)$ as there are $2n^2$ inputs to examine and $n^2$ outputs to compute.
- Such lower bounds are called trivial lower bounds as they are easy to obtain.
- For matrix multiplication, the best known algorithm requires $O(n^{2+E})$

181

# Comparison Trees

- We will discuss use of comparison trees for deriving lower bounds on sorting and searching.
- Sorting problem: Permutation of integers 1 to n, say p(1) to p(n) such that A[p(1)]<A[p(2)]<…..A[p(n)].
- Ordered searching problem: whether a given element x occurs within the elements in A[1:n] which is sorted.
- Merging problem: Two ordered sets of distinct inputs are given. Merge the two sets into a third sorted array.
- Comparison based algorithms can be used to solve these problems where no arithmetic involving elements is permitted.
- Ordered Searching:
  - Comparing x and A[i] can have three outcomes maximum, so maximum two branches in a comparison tree.
  - In the tree atleast n internal nodes corresponding to n different values must be there.

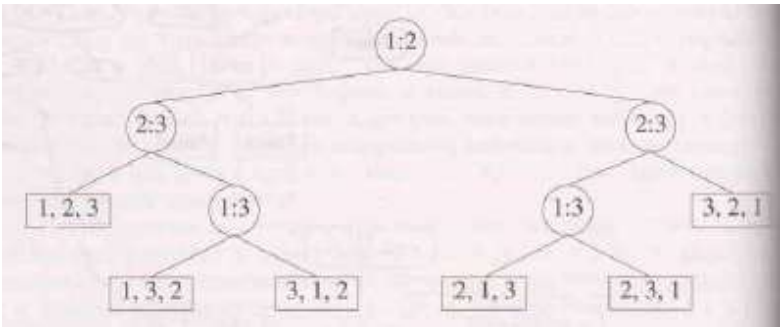
182




183

- Theorem: Let A[1:n] contains n distinct elements which are sorted. Find(n) be minimum number of comparisons needed in the worst case by any algorithm, then Find(n)>=log(n+1)
  - Value of Find(n) is bounded by distance of the longest path from root to leaf node. There are n internal nodes. If all internal nodes of a binary tree are at levels less than or equal to k, then there are at most $2^k$-1 internal nodes. n<=$2^k$-1 and Find(n)=k>=log(n+1)
- Sorting:
  - Suppose all the elements are distinct.
  - Comparison between A[i] and A[j] can have two outcomes: either A[i]<A[j] or A[j]<A[i], so comparison tree is a binary tree. If A[i]<A[j], then left branch is followed, otherwise right branch is followed.
  - Associated with each path from root to an external node is a unique permutation. As there are n! different permutations of n items, so comparison tree must have n! external nodes.

184




- Worst case for all comparison based algorithms:
  - Let T(n) be the minimum number of comparisons sufficient to sort n items in worst case.
  - If all internal nodes are at level k or less, then at most $2^k$ external nodes are there. Let k=T(n) then

    $n! <= 2^{T(n)}$

    $T(n) >= \log(n!)$

    By stirling approximation, log(n!) is nlogn-n/ln 2+(1/2logn +O(1)

    So T(n) if O(nlogn)_

185

# TECHNIQUES FOR ALGEBRAIC PROBLEMS

- Two methods, substitution and linear independence are available for deriving bounds on arithmetic and algebraic problems
- The algebraic problems considered are operations on integers, polynomials and rational functions.
- The model of computation to be used is called a straight line program in which no branching instruction is allowed.
- The only statement in a straight line program is the assignment of the form S: = p op q; S,p,q are variables of bounded size and op is one of the arithmetic operations: addition, subtraction, multiplication or division. p and q are either constants, input variables or variables that have already appeared on the left of an assignment statement.

186

- For eg. – a possible straight line program that computes the value of a 2-degree polynomial $a_2x^2 + a_1x + a_0$ is

  $v_1 = a_2 * x$ ;

  $v_1 = v_1 + a_1$ ;

  $v_1 = v_1 * x$ ;

  $ans = v_1 + a_0$ ;
- Assume that each instruction takes one unit of time and requires one unit of space then complexity of a straight line program is its number of assignments or its length.
- The classes of constants being allowed here are from rings, fields etc.
- Ring: It is an algebraic structure containing a set of elements S and two binary operations denoted by + and *. For each a,b Є S, a+b and a*b are also in S. Also the properties of associativity, commutativity, distributivity, additive identity and multiplicative identity hold in a ring.

187

- Field: A commutative ring such that for each element a in S there is a multiplicative inverse denoted by $a^{-1}$ that satisfies the equation $a * a^{-1}=1$.

188

## Polynomial Evaluation using Straightforward method

- Solving the following polynomial at point v:

  $A(x) = a_n x^n + a_{n-1}x^{n-1} + \ldots..a_0$
- The straightforward method adds $a_1 v$ to $a_0$ and $a_2 v^2$ to this sum and continues as described below :

```
Algorithm Straight Eval (A,n,v)
{ r := 1; s := a0;
    for i := 1 to n do
    {       r := r * v;
            s:= s+ai*r;
    }return s;
}
```

- Straightforward method requires 2n multiplications, n additions, 2n+2 assignments.

189

- An improvement over straightforward evaluation is Horner's rule. A polynomial can be written as:

  $A(x) = (\ldots((a_n x + a_{n-1}) x + a_{n-2}) x + \ldots + a_1) x + a_0$

```
Algorithm Horner(A,n,v)
{  s:= an;
   for i := n-1 to 0 step -1 do
   {       s := s * v + ai;
   }
   return s;
}
```

- The Horner's rule requires n multiplications, n additions and n+1 assignments. Thus, it is an improvement over the straight forward method by a factor of 2.

190

- Theorem: Every algorithm for computing the value of a general $n^{th}$ degree polynomial that uses +, -, and * requires n additions or subtractions.
- Proof: By Straightforward method and Horner's rule. Moreover, if we put x=1 then the equation becomes sum of n+1 terms only that requires n additions or subtractions.
- Another method of proof for deriving lower bounds for algebraic problems is to consider these problems in a matrix setting.
- A polynomial problem can be expressed in the following way : compute the 1 x (n+1) by (n+1) x 1 matrix product:

$$[\,1 \; x \; x^2 \; \ldots\ldots x^n\,] \begin{bmatrix} a_0 \\ a_1 \\ : \\ : \\ a_n \end{bmatrix}$$

which is the product of two vectors

191

## Matrix method for deriving lower bounds for algebraic problems

- For complex number multiplication.
- The product of (a+ib) (c+id) = ac-bd + i(bc+ad) can be written in terms of matrices as :

$$\begin{bmatrix} a & -b \\ b & a \end{bmatrix} \begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} ac - bd \\ bc + ad \end{bmatrix}$$

- Generally we want to consider problems that can be formulated as the product of a matrix times a vector.

192

- Theorem: Let A be an r x s matrix with elements from the extension field F [$x_1$ …,$x_n$ ] and Y = [ $y_1$, …., $y_s$ ] a column vector containing s indeterminates or variables.
  - If the row rank of A is V, then any computation of Ay requires at least V multiplications.
  - If the column rank of A is W, then any computation of Ay requires at least W multiplications.
- Therefore in the evaluation of an nth degree polynomial in terms of a matrix-vector product, n multiplications are necessary as the column rank of the matrix is n.

193

Preconditioning :

- Any algorithm that evaluates a general nth degree polynomial requires n multiplications or divisions and n additions or subtractions. This assertion was based on the assumption that the input into any algorithm was both the value of x plus the coefficients of the polynomial.
- If the coefficients of the polynomial are known in advance and functions of these coefficients can be computed without cost before evaluation begins. This process of computing functions of the coefficients is referred to as preconditioning.

194

- Consider the general fourth-degree polynomial

  $A(x) = a_4x^4 + a_3x^3+a_2x^2+a_1x^1+a_0x^0$

  and the scheme

  $y := (x + c_0) x + c_1$ ; $A(x) := ((y+x+c_2)y+c_3)c_4$;
- Only three multiplications and five additions are required if the values of $c_i$ in terms of $a_i$ can be determined.

$A(x) = c_4x^4 + (2c_0c_4 + c_4) x^3 + (c_0^2 + 2c_1+c_0c_4+c_2c_4) x^2$

$+ (2c_0c_1c_4 + c_1c_4 + c_0c_2c_4) x + (c_1^2c_4+c_1c_2c_4+c_3c_4)$

Equating the above coefficients with the $a_i$

$c_4 = a_4$;

$c_0 = (a_3 / a_4 –1) / 2$

$b = a_2 / a_4-c_0 (c_0+1)$

$c_1 = a_1 / a_4 – c_0b$;

$c_2 = b-2c_1$ ;

$c_3 = a_0 / a_4 – c_1 (c_1 + c_2)$

195

**Example :**

$A(x) = - x^4 + 3x^3 – 2x^2 + 2x + 1$

$a_4 = -1$ ; $a_3 = 3$; $a_2 = -2$; $a_1 = 2$; $a_0 = 1$

$c_4 = -1$ ; $c_0 = -2$; $b = 0$; $c_1 = -2$ ; $c_2 = 4$; $c_3 = 3$

- Applying the scheme

$y := (x+c0) x +c1$ ; $A(x) := ((y + x + c2) y + c3) c4$

the polynomial yields the straight line program

| | |
|---|---|
| $q \leftarrow x –2$ ; | $r \leftarrow q * x$ ; |
| $y \leftarrow r –2$ ; | $s \leftarrow y + x$ ; |
| $t \leftarrow s + 4$ ; | $u \leftarrow t * y$ ; |
| $v \leftarrow u + 3$ ; | $p \leftarrow -1 * v$ ; |

which evaluates to A(x) in just three multiplications.

196

- **Theorem** : A randomly selected polynomial of degree n has probability zero of being computable either with less than $\lceil (n+1) / 2 \rceil$ multiplication/divisions or with less than n additions/ subtractions.
- This theorem shows that the preconditioning method comes very close to being optimal.

197

# Parallel Computations

- Parallel machines offer the potential of decreasing the solution times.
- Eg. If there are 5 loads of clothes to wash and one machine is taking 25 minutes to wash one load. Then total 125 minutes will be required to wash 5 loads. If we use 5 machines only 25 minutes will be sufficient to wash 5 loads.
- For adding 100 numbers, two persons can add 50 numbers individually and then two results can be added to produce the final result.
- It means a problem can be divided into subproblems. One processor can work on each subproblem. When all the processors are done, partial solutions can be combined to produce the final result.

198

- If there are p processors, the solution time can be cut down by a factor of p.
- Algorithm designed for a single processor machine is termed as a sequential algorithm.
- Algorithm designed for a multiprocessor machine is termed as a parallel algorithm.
- For a given problem P, let the best known sequential algorithm has a run time of S'(n). If a parallel algorithm on a p-processor machine runs in time T'(n,P) then the speedup of the parallel algorithm is defined to be S'(n)/T'(n,p).
- If S(n) and T(n,p) are the asymptotic run time of best known sequential algorithm and a parallel algorithm respectively then asymptotic speedup is defined as S(n)/T(n,p). If S(n)/T(n,p) = Θ(p) then algorithm is said to have linear speedup.

199

- Suppose the sequential sorting algorithm has the complexity Θ(nlogn). Let A be an n-processor parallel algorithm that sorts n keys in Θ(logn) time and B be an $n^2$ processor parallel algorithm that also sorts in Θ(logn) time then A has linear speedup but B does not have a linear speedup because for A speedup is Θ(nlogn)/ Θ(logn)=Θ(n) and for B speedup is Θ(nlogn)/Θ(logn)=Θ(n).
- If a p-processor parallel algorithm for a given problem runs in time T(n,p), total work done is defined to be pT(n,p). Efficiency is defined as S(n)/pT(n,p). Parallel algorithm is said to be work optimal if pT(n,p)=O(S(n)).
- Moreover a parallel algorithm is work optimal iff it has linear speedup. Efficiency of work optimal parallel algorithm is Θ(1).
- An important question is 'Is it possible to get a speedup of more than p (superlinear speedup) for any problem on a p-processor machine?
  - If there is a parallel algorithm on a p-processor machine whose speedup is better than p, it means T<S/p ie. pT<S.

200

- A single step of a parallel algorithm can be simulated on a single processor in time <=p, then the whole parallel algorithm can be simulated sequentially in time <=pT<S.
- This is contradiction as S is the run time of the best known sequential algorithm.
- This result is valid only when we consider asymptotic speedups. It may not be valid when actual run times are being considered because.

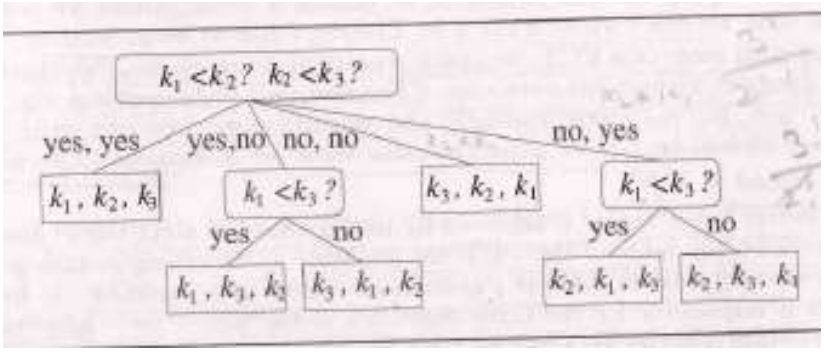
- To achieve a good speedup, it is necessary to parallelize every component of the underlying technique. If a fraction cannot be parallelized then maximum speedup is limited by f
- According to Amdahl's Law:
  - Maximum Speedup = 1/(f+((1-f)/p).

201

# Lower Bounds on Parallel Computations

- Parallel model assumed is Parallel Comparison Tree (PCT).
- A PCT with p processors is a tree wherein at each node, at most p pairs of comparisons are made. Depending on the outcomes, the computation proceeds to an appropriate child.
- In sequential comparison tree, each node can have at most two children. In PCT, the number of children depends on p.
- In sorting problem, as there can be n! different possible permutations of n items, a PCT must have atleast n! external nodes.
- Eg. A PCT with two processor that sorts three numbers k1,k2,k3. Two comparisons can be performed at each node. The depth of this PCT is two which is the worst case run time of the parallel algorithm.

202




- Worst case time of any algorithm on PCT is the maximum depth of any external node. Average case considers average depth.
- It can be concluded that any comparison problem can be solved in O(1) time given enough processors.

203

Lower Bound on Average case sorting:

- If p >= ${}^nC_2$, sorting can be done in O(1) time on a PCT. If p< ${}^nC_2$ then two lemmas:
  - Lemma(Shannon): A tree with degree d and l external nodes has an average depth of at least log l/log d.
  - l has to be atleast n!.
  - At each node of a PCT, we make p comparisons. So there can be as many as $2^p$ possible outcomes.
  - If we substitute this value for d then lower bound is nlogn/p.
  - A better lower bound is achieved by noting that not all the $2^p$ possible outcomes are feasible. For eg. For comparing 3 numbers x,y,z; x>y, y>z and x>z, yes, yes, no is not possible.

204

- A graph can be introduced, which has n nodes, one node per input number. For each node v in the PCT, graph Gv is made. For each comparison x>y in node v in PCT, we draw an edge between x and y. So Gv has p edges and is undirected. We can orient each edge of Gv depending on the outcome of the comparison. Degree of v is the number of ways in which we can orient the p edges of Gv.
- As the input numbers are from a linear order, any orientation of the edges of Gv that introduces a directed cycle is impossible. So the number of acyclic orientations are better estimate of d.

- Lemma: A graph with n vertices and m edges has at most $(1+2m/n)^n$ acyclic orientations.
- So by putting $d=(1+2p/n)^n$, we can calculate the depth log l/log d where l=n! and get a lower bound = Ω(logn/(log(1+p/n))).

205

# NP-HARD/COMPLETE PROBLEMS

## NP –HARD AND NP – COMPLETE PROBLEMS

- Under this topic we are concerned with distinction between problems that can be solved by a polynomial time algorithm and problems for which no polynomial time algorithm is known.
- Problems can be divided into two groups: Group1– problems for which solution can be found in polynomial time ie time complexity is bound by a polynomial of small degree. For eg. O(n), O(mn)

  Group2 – problems for which best known algorithms are non-polynomial.
- Theory of NP-completeness does not provide method of obtaining polynomial time algorithms nor does it proves something. It only shows that many of the problems for which there are no known polynomial time algorithms are computationally related.

207

- A problem which is NP complete has the property that it can be solved in polynomial time iff all other NP – complete problems can be solved in polynomial time.
- If an NP-hard problem can be solved in polynomial time, then all NP-complete problems can be solved in polynomial time.
- All NP-complete problems are NP-hard but some NP-hard problems are not known to be NP-complete.
- Conclusion : No NP-complete or NP-hard problem is polynomially solvable.

208

## DETERMINISTIC and NONDETERMINISTIC ALGORITHMS

- Algorithms having the property that the result of every operation is uniquely defined are called deterministic algorithms. Programs are executed on a computer in this way.
- In theory, some algorithms are allowed that contain operations whose outcome is not uniquely defined but is limited to a specified set of possibilities. Such algorithms are called non-deterministic algorithms.
- The machine executing such operations are allowed to choose any one of these outcomes subject to a termination condition.
- Following three functions are used in non-deterministic algorithms
  - Choice(S) : arbitrarily chooses one of the elements of set S. There is no rule specifying how this choice is to be made.
  - Failure() : signals an unsuccessful completion.
  - Success() : signals a successful completion.

209

- A non-deterministic algorithm terminates unsuccessfully if and only if there exists no set of choices leading to a success signal.
- Computing times for Choice, Success and Failure are taken to be O(1).
- A machine capable of executing a nondeterministic algorithm is called a non-deterministic machine which do not exist in practice.
- Example1: Searching an element x in an array A[1:n]
  1. j:=Choice(1,n)
  2. If A[j]=x then { write (j); Success();}
  3. write(0); Failure();

  Complexity of this algorithm is O(1). Every deterministic search algorithm is of complexity Ω(n).

210

- Example2: Sorting in increasing order.

```
Algorithm Nsort(A,n)
{   for i:=1 to n do B[i]:=0;
    for i:=1 to n do
    {       j:=Choice(1,n);
            if B[j]≠0 then Failure();
            B[j]:=A[i];
    }
    for i:=1 to n-1 do
            if B[i]>B[i+1] then Failure();
    write (B[1:n]);
    Success();
}
```

Its complexity is O(n). All deterministic sorting algorithms have complexity Ω(nlogn)

211

- A deterministic interpretation of a non-deterministic algorithm can be made by allowing unbounded parallelism in computation. Each time a choice is to be made, algorithms makes several copies of itself. One copy for each of the possible choices. First copy to reach successful completion terminates all other computations. If a copy reaches failure then only that copy terminates.
- A non-deterministic machine does not make any copies of an algorithm every time a choice is to be made.
- Any problem for which the answer is either zero or one is called decision problem.
- An algorithm for a decision problem is termed as a decision algorithm.
- Any problem that involves the identification of an optimal value of a given cost function is known as an optimization problem.
- An optimization algorithm is used to solve an optimization problem.

212

- In non-deterministic decision algorithms, a successful completion is made if and only if the output is1. A 0 is output if and only if there is no sequence of choices leading to a successful completion.
- Many optimization problems can be recast into decision problems with the property that the decision problem can be solved in polynomial time iff the corresponding optimization problem can.
- In other cases, if the decision problem cannot be solved in polynomial time, then the optimization problem cannot either.
- Example3: Maximum Clique:
  - A maximal complete subgraph of a graph G=(V,E) is a clique. Size of clique is the number of vertices in it.
  - Max Clique problem is an optimization problem that has to determine the size of largest clique in G. Corresponding decision problem is to determine whether G has a clique of size k. Suppose DClique(G,k) is a deterministic decision algorithm for Clique decision problem. If n is the number of vertices in G, then size of max clique can be found by executing Dclique for each value of k=n, n-1, …….. If time complexity of DClique is f(n) then optimization problem can be solved in time <=nf(n). Also if the optimization problem can be solved in time g(n), then decision problem can also be solved in time g(n). So optimization problem can be solved in polynomial time iff decision problem can be solved in polynomial time.

213

- Non-deterministic Algorithm for Clique decision problem:

```
Algorithm DCK(G,n,k)
{       S:=Ø;
        for i:=1 to k do
        {           t:=Choice(1,n);
                    if t Є S then Failure();
                    S:=S U {t};
        }
        for all pairs (i,j) such that i Є S, j Є S and i ≠ j do
                    if (i,j) is not an edge of G then Failure();
        Success();
}
```

- Example4: 0/1 Knapsack: Decision problem is to determine whether there is a 0/1 assignment of values to $x_i$ such that $\sum p_i x_i >= r$ and $\sum w_i x_i <= m$. If decision problem cannot be solved in deterministic polynomial time, then the optimization problem cannot either.

214

- Non-deterministic Knapsack Algorithm:

```
Algorithm DKP (p,w,n,m,r,x)
{       W:=0; P:=0;
        for i:=1 to n do
        {       x[i] := Choice(0,1);
                W:= W+x[i]*w[i];
                P:=P+x[i]*p[i];
        }
        if ((W>m) or (P<r)) the Failure();
        else Success();
}
```

- Time required by a nondeterministic algorithm is the minimum number of steps needed o reach a successful completion if there exists a sequence of choices leading to successful completion. In case successful completion is not possible, then time required is O(1).

215

- Deterministic sum of subsets algorithm:

```
Algorithm Sum(A,n,m)
{       s:=1;//s is an m+1 bit word. Bit zero is 1.
        for i:=1 to n do
                s:=s or Shift(s,A[i]);.//Shift shifts the bits to left.
        if the mth bit in s is 1 then
                write (“A subset sums to m.”);
        else
                write (“No subset sums to m.”);
}
```

Complexity of this algorithm is O(mn)

216

# SATISFIABILITY

- Let $x_1, x_2, x_3 \ldots x_n$ denotes Boolean variables.
- Let $\bar{x}_i$ denotes the negation of $x_i$.
- A literal is either a variable or its negation.
- A formula in the prepositional calculus is an expression that can be constructed using literals and the operators and $\Lambda$ or V.
- A formula is in **conjunctive normal form** (CNF) if it is a conjunction of clauses, where a clause is a disjunction of literals.
- A formula is in **disjunctive normal form** (DNF) if it is a disjunction of clauses, where a clause is a conjunction of literals.
- The satisfiability problem is to determine if a formula is true for some assignment of truth values to the variables.
- CNF-Satisfiability is the satisfiability problem for CNF formulas.

217

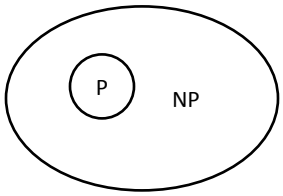
- Non-deterministic Satisfiability

```
Algorithm Eval(E,n)
{     for i:=1 to n do
            x_i := Choice (false, true);
      if E(x_1,x_2,…x_n) then Success();
      else Failure();
}
```

218

## Classes NP-hard and NP-complete

- P is the set of all decision problems solvable by deterministic algorithms in polynomial time.
- NP is the set of all decision problems solvable by nondeterministic algorithms in polynomial time.
- Since deterministic algorithm are a special case of non deterministic algorithms, we can conclude that $P \subseteq NP$.
- The most famous unsolved problem in computer science is whether P = NP or $P \neq NP$. Is it possible that for all the problems in NP there exist polynomial time deterministic algorithms which have remained undiscovered ? Till now assumption is $P \neq NP$.

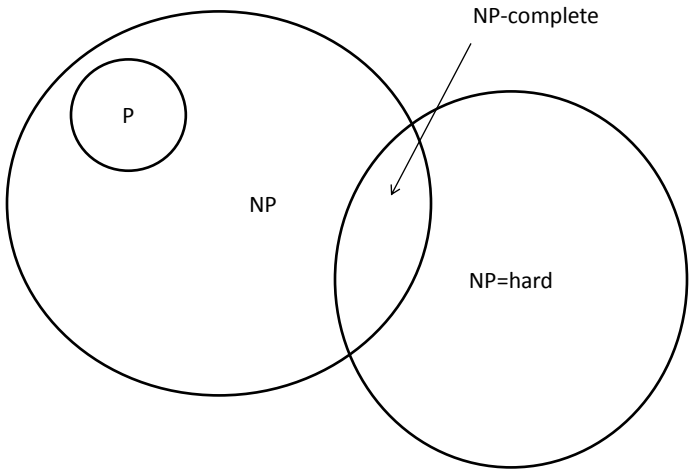



219

- S.COOK formulated the following question :
- Is there any single problem in NP such that if we showed it to be in P then that would imply that P = NP? Cook answered his own question in positive.
- **COOK's Theorem** : Satisfiability is in P if and only if P = NP.
- **Reducibility**
  - Let $L_1$ and $L_2$ be problems. $L_1$ reduces to $L_2$ ($L_1 \alpha L_2$) if and only if there is a way to solve $L_1$ by a deterministic polynomial time algorithm using a deterministic algorithm that solves $L_2$ in polynomial time.
  - If $L_1 \alpha L_2$ and $L_2 \alpha L_3$ then $L_1 \alpha L_3$.
- NP-Hard Problem : A problem L is NP-hard if any only if satisfiability reduces to L (satisfiability $\alpha$ L).
- NP-Complete: A problem is NP-complete if an only if L is NP-hard and $L \in NP$.

220

- Only a decision problem can be NP-complete. An optimization problem may be NP-hard.




221

- **Example :**
- Halting problem is NP-hard decision problem, but it is not NP-complete.

**Halting Problem** :

- To determine for an arbitrary deterministic algorithm A and an input I whether algorithm A with input I ever terminates (or enters an infinite loop).
- Halting problem is un-decidable.
- Hence there exists no algorithm to solve this problem.
  - So, it is not in NP.
  - So, it is not NP-complete.
- To show that Halting problem is NP-hard, we show that satisfiability $\alpha$ halting problem.
- For this let us construct an algorithm A whose input is a prepositional formula X. Suppose X has n variables. Algorithm A tries out all $2^n$ possible truth assignments and verifies if X is satisfiable.

222

- If it is then A stops.
- If X is not satisfiable, then A enters an infinite loop.
- Hence A halts on input iff X is satisfiable.
- If we had a polynomial time algorithm for the halting problem, then we could solve the satisfiability problem in polynomial time using A and X as input to the algorithm for the halting problem
- Hence satisfiability $\alpha$ halting problem.
- Hence the halting problem is an NP-hard problem which is not in NP
- Thus Halting problem is NP-hard, but not NP-complete

223

## NP-HARD GRAPH PROBLEMS

- The strategy to show that a problem $L_2$ is NP-hard is
  - Pick a problem $L_1$ already known to be NP-hard.
  - Show how to obtain (in polynomial deterministic time) an instance I' of $L_2$ from any instance I of $L_1$ such that from the solution of I' we can determine (in polynomial deterministic time) the solution to instance I of $L_1$.
  - Conclude that $L_1 \alpha L_2$.
  - As L1 is NP hard, Satisfiability $\alpha$ L1 and L1 $\alpha$ L2,
  - $\therefore$ Satisfiability $\alpha$ L2
  - Hence $L_2$ is NP-hard.
- An NP-hard decision problem $L_2$ can be shown to be NP-complete by exhibiting a polynomial time non-deterministic algorithm for $L_2$.

224

- Clique Decision Problem (CDP)
  - We will show that CNF-satisfiability α CDP.
  - Let $F = C_1 \Lambda C_2 \Lambda C_3 \ldots \Lambda C_k$ be a propositional formula in CNF.
  - Let $x_1, x_2, \ldots x_n$ be variables in F.
  - We can construct a graph G=(V,E) from F such that G has a clique of size k if and only if F is satisfiable.
  - For any F, G =(V,E) is defined as
    - $V=\{<\sigma,i> \mid \sigma$ is a literal in clause $C_i\}$
    - $E=\{(<\sigma,i>,<\delta,j>) \mid i \neq j$ and $\sigma \neq \delta$.
  - F is satisfiable iff G has a clique of size k.
  - If F is satisfiable, then there is atleast one literal σ in each $C_i$ such that σ is true. Let $S = \{< \sigma,i> \mid \sigma$ is true in $C_i\}$ be a set containing exactly one $< \sigma,i>$ for each i.
  - Between any two nodes $< \sigma,i>$ and $< \delta,j>$ in S there is an edge in G since $i \neq j$ and both σ and δ have value true. Thus S forms a clique of size k.

225

  - Similarly if G has a clique K = (V',E') of size k then let $S=\{<\sigma,i> \mid <\sigma,i> \in V'\}$.
  - If $S' = \{\sigma \mid <\sigma,i> \in S$ for some i} then S' cannot contain both a literal δ and its complement as there is no edge connecting $<\delta,i>$ and $<\delta,j>$.
  - So by setting $x_i$ = true if $x_i \in S'$ and $x_i$=false if $x_i \notin S'$ and choosing arbitrary truth values for variables not in S' we can satisfy all clauses in F.
  - Hence F is satisfiable iff G has a clique of size k.
  - CNF-satisfiability $\alpha$ CDP, hence CDP is NP-hard. CDP ∈ NP, so CDP is also NP-complete.

226

- Node Cover Decision Problem (NCDP):
  - A set $S \subseteq V$ is a node cover for a Graph = (V,E) iff all edges in E are incident to at least one vertex in S. Size of the cover is the number of vertices in the set.
  - Decision problem is to determine whether G has a node cover of size k?
  - We will show that CDP α NCDP. As Satisfiability α CDP and CDP α NCDP so satisfiability α NCDP and hence NCDP is NP hard.
  - Moreover we can easily design non-deterministic algorithm that can verify in polynomial time whether S is a cover for G. Hence NCDP is also NP-complete.
  - To prove that CDP α NCDP:
    - Let G=(V,E) be the graph having n vertices and it has a clique of size k. From G, we construct G' that has a node cover of size at most n-k if and only if G has a clique of size at least k.

227

- Graph G'=(V,E') where E'={(u,v)|uЄV and vЄV and $(u,v) \notin E$}. G' is the complement of G.
- Let k be the clique in G. Since there are no edges in E' connecting vertices in K, the remaining n-|k| vertices in G' must cover all edges in E'
- Similarly if S is the node cover in G' then V-S must form a complete subgraph in G.

228

- Chromatic Number Decision Problem (CNDP)
  - SATY (Satisfiability with at most three literals per clause) α CNDP
  - Let F be a CNF formula having at most three literals per clause and having r clauses $C_1, C_2, \ldots C_r$. Let $x_i$ be the n variables in F.
  - Assume that n>=4. For n<4, it is very simple to evaluate the formula for 8 possible values of $x_i$.
  - From F we can construct in polynomial time a graph G that is n+1 colorable iff F is satisfiable. Graph G = (V,E) is defined as:
    - $V=\{x_1,x_2,\ldots x_n\} U \{x_1',x_2',\ldots x_n'\} U \{y_1,y_2,\ldots y_n\} U \{C_1,C_2,\ldots C_r\}$ where $y_1,y_2,\ldots y_n$ are new variable.
    - $E=\{(x_i,x_i'), 1<=i<=n\} U \{(y_i,y_j) \mid i \neq j\} U \{(y_i,x_j) \mid i \neq j\} U \{(y_i,x_j') \mid i \neq j\} U \{(x_i,C_j) \mid xi \notin C_j\} U \{x_i',C_j) \mid x_i' \notin C_j\}$
    - $y_i$ form a complete subgraph on n vertices. Hence each $y_i$ must be assigned a distinct color. Suppose $y_i$ is given the color i. $y_i$ is connected to all $x_j$ and $x_j'$ except $x_i$ and $x_i'$, the color i can be assigned to only $x_i$ and $x_i'$. But $x_i$ and $x_i'$ are also connected, so a new color n+1 is needed for one of these vertices.
    - Suppose the vertex which is assigned the new color n+1 is the false vertex and the other one is a true vertex.

229

- To color G using n+1 colors, it is required to assign color n+1 to one of $x_i$, $x_i'$ for each i.
- Now how to color the remaining vertices $\{C_i\}$ without using any new color?
- As n>=4 and each $C_i$ has at most three literals, each $C_i$ is adjacent to a pair of vertices $x_j$, $x_j'$ for atleast one j. So no $C_i$ can be assigned the color n+1.
- Moreover no $C_i$ can be assigned a color corresponding to an $x_j$ or $x_j'$ not in the clause $C_i$.
- So the only colors that can be assigned to $C_i$ correspond to vertices $x_j$ or $x_j'$ which are in the clause $C_i$ and are true values.
- So G is n+1 colorable if and only if there is a true vertex corresponding to each $C_i$ ie. if F is satisfiable.

230

- Directed Hamiltonian Cycle (DHC)
  - A hamiltonian cycle goes through every vertex in the Graph exactly once and then returns to the starting vertex.
  - CNF-satisfiability α DHC
  - F is the propositional formula in CNF. We will construct a directed graph G such that F is satisfiable iff G has directed hamiltonian cycle.
  - Construction of graph for example F=C1ΛC2 ΛC3 ΛC4 where C1= x1Vx2'Vx4Vx5', C2=x1'Vx2Vx3, C3=x1'Vx3'Vx5, C4= x1'Vx2'Vx3'Vx4Vx5'.
  - If F is satisfiable then G has a hamiltonian cycle.
- Travelling Salesperson Problem (TSP)
  - DHC α TSP
  - From directed graph G=(V,E) construct a complete directed graph G'=(V,E') and c(i,j)=1 if <i,j> Є E otherwise c(I,j)=2.
  - G' has a tour of cost n iff G has a directed Hamiltonian cycle.

231

- AND/OR Graph Decision Problem(AOG)
  - Complex problems can be divided into series of subproblems such that solution of all or some of these results in the solution of original problem. This is called problem reduction. This structure can be represented by using And-Or graph.
  - Terminal nodes in the graph represents primitive problems are are marked either solvable or not solvable.
  - Cost is associated with each edge in the AOG. Cost of the solution graph is the sum of the costs of the edges in that graph.
  - AOG decision problem is to determine whether G has a solution graph of cost at most k.
  - We will show that CNF-satisfiability α AOG ie. AOG is NP-hard.
  - P has k clauses in n variable. An AOG can be constructed from P such that AOG has a cost of n iff P is satisfiable.

232

# NP Hard Scheduling Problems

- Partition Problem: Whether a given set A={$a_1$,$a_2$,…$a_n$) of n positive integers has a partition P such that $\sum_{(i\in P)}a_i = \sum_{(i\notin P)}a_i$. This problem is NP Hard.
- Exact cover problem: Let F = {$S_j$} be a family of sets. T is a set cover of F iff $\cup_{(Si\in T)}S_i = \cup_{(Si\in F)}S_i$ . T is an exact cover of F if sets in T are disjoint.
- Exact cover problem $\propto$ sum of subsets problem
- Sum of Subsets $\propto$ partition problem.
- Exact cover problem is NP hard, so partition problem is also NP Hard.

233

- Scheduling Identical Processors:
  - Let Pi be m identical processors. Let $J_i$ be n jobs. Job Ji requires ti processing time. A schedule S is an assignment of jobs to processors. A job cannot be processed by more than one processor at any given time. Let fi be the time at which the processing of job Ji is completed. Mean Finish Time is:
    - $MFT(S)=1/n\sum_{(i=1to\ n)}fi$
  - If wi is the weight associated with each job then Weighted mean finish time is:
    - $WMFT(S)= 1/n\sum_{(i=1to\ n)}wifi$
  - Let Ti be the time at which Pi finishes processing all jobs. Finish time of S:
    - $FT(S) = max_{(i=1\ to\ m)}\{Ti\}$
  - Partition $\propto$ minimum finish time non preemptive schedule.
  - Partition $\propto$ minimum WMFT non preemptive schedule.

234

- Flow Shop Scheduling:
  - Let there be n jobs each requiring m tasks $T_{1i}, T_{2i}, T_{3i}, \ldots T_{mi}$ where i varies from 1 to n. Task $T_{ji}$ is to be performed on processor $P_j$ where j varies from 1 to m. Time required to complete task $T_{ji}$ is $t_{ji}$. A schedule of n jobs is an assignment of tasks to time intervals on the processors. Task $T_{ji}$ must be assigned to processor $P_j$. For any job i the processing of task $T_{ji}$, j>1 cannot be started until task $T_{j-1,i}$ has been completed.
  - Partition $\propto$ minimum finish time preemptive flow shop schedule (m>2)

235

- Job Shop Scheduling:
  - There are m different processors. The n jobs to be scheduled require the completion of several tasks. Time of $j^{th}$ task for job $J_i$ is $t_{k,i,j}$ ie. Task j of ith job is to be performed on processor $P_k$. The tasks for any job are to be carried out in the order. Many tasks of the same job can be processed on the same processor.
  - Partition $\propto$ minimum finish time preemptive job shop schedule(m>1)

236

# Some Simplified NP hard problems

- A problem which is NP hard can not be solved in deterministic polynomial time.
- Some restricted versions of NP hard problems can be solved in deterministic polynomial time.
- Examples:
  - CNF satisfiability with at most three literals per clause is NP hard. If each clause is restricted to have at most two literals, then is problem is polynomially solvable.
  - Determining whether a planar graph is three colorable is NP hard. To determine whether it is two colorable, we only have to see whether it is bipartite.

237